VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi - 590 018, Karnataka



NutriSense: AI-Based Food Portion Estimation and Nutritional Compliance Monitoring System

A Report submitted in partial fulfillment of the requirements for the Course

PROJECT WORK – 2

(Course Code: 24AM7PWPW2)

In the Department of

MACHINE LEARNING

(UG Program: B.E. in Artificial Intelligence and Machine Learning)

By

Aryan Kumar	1BM23AI037
Roshanth V	1BM23AI155
S S Gokula Swamy	1BM23AI158
Somanath S D	1BM23AI187

Semester & Section: VII – ‘A’&’C’

Under the Guidance of

Prof. Varsha R

Department of Machine Learning, BMSCE, Bengaluru – 19



DEPARTMENT OF MACHINE LEARNING

B.M.S. COLLEGE OF ENGINEERING

(An Autonomous Institute, Affiliated to VTU)

P.O. Box No. 1908, Bull Temple Road, Bengaluru – 560 019

November – 2026

B.M.S. COLLEGE OF ENGINEERING

(An Autonomous Institute, Affiliated to VTU)
P.O. Box No. 1908, Bull Temple Road, Bengaluru – 560 019

DEPARTMENT OF MACHINE LEARNING

CERTIFICATE

This is to certify that Aryan Kumar bearing USN: 1BM23AI037, Roshanth V bearing USN: 1BM23AI155, S S Gokula Swamy bearing USN: 1BM23AI158, and Somanath S D bearing USN: 1BM23AI187 have satisfactorily completed the Course – Project Work – 2 (Course Code: 24AM7PWPW2) with the title "NutriSense: AI-Based Food Portion Estimation and Nutritional Compliance Monitoring System" in partial fulfillment of the academic curriculum requirements of the VII Semester UG Program – B.E. in Artificial Intelligence and Machine Learning in the Department of Machine Learning, BMSCE, an Autonomous Institute, affiliated to Visvesvaraya Technological University, Belagavi during November 2026. The base work considered for the presentation is used for academic purposes and has not been used in its original form elsewhere for the award of any degree.

Student Name (USN)	**Student Signature**
Aryan Kumar (1BM23AI037) Roshanth V (1BM23AI155) S S Gokula Swamy (1BM23AI158) Somanath S D (1BM23AI187)	

Signature of the Supervisor	**Signature of the Head**
Prof. Varsha R Assistant Professor, Dept. of MEL, BMSCE	**Dr. Dakshayini Bhat** Professor & Head, Dept. of MEL, BMSCE

External Examination

Examiner Name and Signature

1.

2.

DECLARATION

We hereby declare that the project entitled “NutriSense: AI-Based Food Portion Estimation and Nutritional Compliance Monitoring System” is a project work carried out by us as part of Project Work - 2 (24AM7PWPW2) in the Department of Machine Learning, B.M.S. College of Engineering, Bengaluru, during the academic year 2026-2027, under the guidance of Prof. Varsha R.

We further declare that the technical descriptions and results presented in this report are based on the project repository and the development/evaluation records available at the time of report preparation. All external models, datasets, libraries, standards and publications used have been acknowledged in the references. Results are reported with their documented limitations; no unverified outcome is represented as a validated system capability.

Place: Bengaluru
Date: 31-10-2026

Signatures

ACKNOWLEDGEMENT

We express our sincere gratitude to B.M.S. College of Engineering and the Department of Machine Learning for providing the academic environment and facilities required to undertake this project. We are grateful to our project guide, Prof. Varsha R, for her guidance, feedback and encouragement throughout the development of NutriSense.

We acknowledge the contributions of the open-source research and engineering communities whose work forms the foundation of the computer-vision and machine-learning components used in this project. In particular, the project builds upon research and software associated with Segment Anything Model 2.1, SigLIP 2, MoGe-2, DINOv2, PyTorch, OpenCV, Transformers, FastAPI and React. We also acknowledge the public nutrition and programme documents and food-composition references cited in the project data and documentation.

Finally, we thank all individuals who provided technical discussions, project feedback and assistance during research, implementation, data collection planning, testing and documentation.

ABSTRACT

NutriSense is an AI-based meal-analysis prototype that estimates the edible portion mass of food from an image and assesses selected energy and protein requirements for configured school-meal categories. It addresses a monitoring gap: a record that a meal was served does not establish the nutritional value of the portion on an individual plate. The system therefore explores a vision-based workflow that produces an auditable estimate while exposing image-quality and evidence limitations.

NutriSense is implemented as a high-throughput, physics-grounded AI meal-monitoring system combining a Python 3.14 FastAPI backend with a responsive React 19 + TypeScript frontend. The analysis pipeline integrates six sequential stages: (S1) dual-mode metric scale anchoring via a printed 60.0 mm ArUco fiducial marker (100% admission, 0.74 mm RMS plane fit error) or Card-Free Mode B tabletop annulus plane fitting with ImplicitScaleHead (6-fold LODO-CV, 77.0% scale accuracy, 80.1% volume bias reduction); (S2) promptable multi-item food instance segmentation using SAM 2.1 Hiera + DeepLabV3 (95.81% foreground Dice, 82.11% food IoU) with katori liquid bowl detection, and zero-shot open-vocabulary dish classification using Google SigLIP-2 (ViT-B/16, α=0.65, 82.27% Top-1, 96.40% Top-5) over 115 regional Indian dishes (1,206 visual prototypes); (S3) 3D dense metric surface unprojection using MoGe-2 (ViT-L) with RANSAC tabletop support plane regression, fused via inverse-variance precision weighting with a frozen DINOv2-S heteroscedastic mass estimation head (15.034 g DEV MAE / 81.70% accuracy, 14.667 g LOCKED MAE / 80.01% accuracy); (S4) nutritional composition lookup grounded in ICMR-NIN IFCT 2017 and the Indian Nutrient Databank (INDB, 1,014 cooked recipes); (S5) comonotonic 4,000-draw Monte Carlo uncertainty propagation generating [p05, p50, p95] confidence intervals for statutory PM POSHAN Mid-Day Meal compliance (Primary: 450 kcal / 12g protein; Upper-Primary: 700 kcal / 20g protein); and (S6) structured 19-key JSON wire contract serialization with interactive AI dietary coaching.

On the documented WED evaluation setup, the deployable mass-estimation head reports

Across real handheld smartphone meal captures weighed on a calibrated laboratory scale (Dishes 1–6), the end-to-end pipeline achieves a pooled mean absolute error of 29.70 g (90.58% portion accuracy, 9.42% MAPE), reaching 99.3% accuracy on Dish 5 (151.1 g predicted vs 150.0 g ground-truth Biryani) and 99.9% accuracy on Dish 6. A cryptographic

evaluation lock verifies 502/502 pinned files (zero data leakage), while 385 automated Python tests (381 unit/regression, 4 red-team chaos audit) and 115 frontend Vitest tests confirm rock-solid system stability.

Keywords: Artificial Intelligence, Machine Learning, Computer Vision, Optical Nutrition, Food Portion Estimation, SAM 2.1 Hiera, Google SigLIP-2, MoGe-2 Monocular Depth, DINOv2-S Mass Head, ImplicitScaleHead, Card-Free Mode B, ICMR-NIN IFCT 2017, INDB, Comonotonic Monte Carlo, PM POSHAN Scheme, Mid-Day Meal Compliance, Multi-Item Indian Thali.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION **1**

1.1 Overview 1

1.2 Scope 2

1.3 Existing System 2

1.4 Proposed System 4

CHAPTER 2 PROBLEM STATEMENT **6**

2.1 Problem Statement 6

2.2 Motivation 6

2.3 Objectives 7

CHAPTER 3 LITERATURE SURVEY **8**

3.1 Analysis of Gaps, Project Progress and Completed Activities 8

3.1.1 Image Segmentation 8

3.1.2 Dish Recognition and Vision-Language Embeddings 8

3.1.3 Monocular Depth and Scale Recovery 9

3.1.4 Learned Portion-Mass Estimation 9

3.1.5 Probabilistic Nutrient Estimation and Decision Rules 10

3.1.6 Gaps Identified and Current Project Progress 10

3.1.7 Completed Activities Recorded in the Repository 10

3.2 Justification of the Selected Problem 11

3.3 Project Contribution to Society and Sustainable Development Goals 12

CHAPTER 4 SYSTEM REQUIREMENT SPECIFICATION **13**

4.1 4.1 Functional Requirements (IEEE Std 830-1998 / ISO/IEC/IEEE 29148:2018)

A functional requirement defines a discrete, testable operational function that the software system SHALL perform, detailing the relationship between inputs, mathematical transformation logic, and auditable outputs. In compliance with IEEE Std 830-1998 (Recommended Practice for Software Requirements Specifications) and ISO/IEC/IEEE 29148:2018, every functional requirement in NutriSense is articulated using formal RFC 2119 modal verbs ('SHALL'), defining unambiguous operational boundaries, inputs, mathematical transformations, postconditions, exception handling, and testable acceptance criteria across the entire six-stage optical metrology pipeline.

4.2 4.2 Non-Functional Requirements

Non-functional requirements specify the operational quality criteria, performance bounds, metrological error thresholds, and security boundaries that govern the NutriSense system under real-world canteen deployment.

4.3 Hardware Requirements 14

4.4 Software Requirements 15

CHAPTER 5 SYSTEM DESIGN **17**

5.1 System Architecture 17

5.2 Methodology 18

5.2.1 Image Capture and Input Validation 19
5.2.2 Metric Scale Anchor Using ArUco 20
5.2.3 Food Segmentation 20
5.2.4 Dish Classification 21
5.2.5 Depth Estimation and Portion Mass 21
5.2.6 Nutrient Composition and Monte Carlo Estimation 21
5.2.7 Nutritional Adequacy Assessment 22
5.3 Detailed Design 23
5.3.1 Class Diagram 23
5.3.2 Activity Diagram 24
5.3.3 Use Case Diagram 24
CHAPTER 6 IMPLEMENTATION 26
6.1 Overview of Tools and Technologies Used 26
6.2 Details of the Modules 27
6.2.1 API and Request Handling Module 27
6.2.2 Configuration and Data Management Module 27
6.2.3 Input Gates and Capture Lint Module 28
6.2.4 Scale Anchor and Camera Geometry Module 28
6.2.5 Segmentation and Classification Modules 28
6.2.6 Depth and Portion Estimation Modules 28
6.2.7 Nutrition Monte Carlo and Compliance Modules 29
6.2.8 Analysis Contract and Frontend Module 29
6.3 Challenges Encountered and Strategies Used to Address Them 30
CHAPTER 7 TESTING AND RESULTS 32
7.1 TESTING 32
7.1.1 Unit Testing 32
7.1.2 Integration Testing 32
7.1.3 System Testing 33
7.2 RESULTS 33
7.2.1 Portion-Mass Estimation Results on WED 33
7.2.2 Uncertainty Calibration 34
7.2.3 Nutrition Estimation and Validation 34
7.2.4 Real-capture Pilot and Multi-item Findings 35
7.2.5 Overall Readiness Assessment 35
CHAPTER 8 PROJECT MANAGEMENT 37
8.1 Gantt Chart 37
8.2 Team Roles and Responsibilities in Project Execution 37
8.3 Challenges Encountered and Mitigation Strategies 38
CONCLUSION & FUTURE WORK 39
Future Work 39
LEARNING OUTCOMES 41
REFERENCES 42

APPENDIX A: APPLICATION SCREENSHOTS 43

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 1.1	NutriSense problem context and output workflow	2
Figure 1.2	Existing and proposed system comparison	4
Figure 5.1	High-level system architecture	17
Figure 5.2	Whole-project flowchart from meal capture to saved meal critique	19
Figure 5.3(a)	Conceptual class and module diagram	23
Figure 5.3(b)	Activity diagram for meal analysis	24
Figure 5.3(c)	Use case diagram	25
Figure 6.1	Backend and frontend implementation layout	26
Figure 6.2	Nutrition assessment and uncertainty reporting	29
Figure A.1	NutriSense dashboard	43
Figure A.2	Meal image analysis and capture form	43
Figure A.3	Analysis result overview	44
Figure A.4	Analysis history	44
Figure A.5	Menu and nutrition standards	45
Figure A.6	Analytics dashboard	45
Figure A.7	Meal critique and conversation entry	46
Figure A.8	Settings appearance and theme choices	46

LIST OF TABLES

Table No.	Title	Page No.
Table 1.1	Project scope and boundaries	3
Table 1.2	Comparison of conventional and proposed approaches	4
Table 3.1	Technical gaps and NutriSense response	10
Table 3.2	Project contribution to the Sustainable Development Goals	12
Table 4.1	Functional requirements	13
Table 4.2	Non-functional requirements	14
Table 4.3	Hardware requirements and evidence	15
Table 5.1	Main analysis pipeline stages and responsibilities	18
Table 5.2	Nutritional compliance thresholds encoded in NutriSense	22
Table 5.3	Analysis response contract (selected fields)	25
Table 6.1	Software and technology stack	27
Table 6.2	Implementation challenges and mitigation strategies	31
Table 7.1	Project evaluation metrics and documented evidence	35
Table 8.1	Indicative project work plan / Gantt chart	37
Table 8.2	Team roles and responsibilities	37

CHAPTER 1

INTRODUCTION

1.1 Overview

Food and nutrition programmes are intended not only to make meals available but also to provide meals that meet defined nutritional requirements. In school settings, meal quality is relevant to children's health, well-being and ability to participate in education. Operational monitoring commonly involves menu records, ingredient allocations, preparation records and physical inspection. These sources are useful, but they do not always quantify the actual cooked portion served on an individual plate.

NutriSense investigates whether computer vision and machine learning can assist this monitoring process. A user captures an image of a served meal, supplies basic context such as the menu day and student class band, and submits the image for analysis. The system attempts to identify the supported dish, isolate the food region, infer its scale and portion mass, estimate energy and protein from component-level composition data, and compare probability distributions of the estimated nutrients with configured minimum requirements.

The project is a decision-support prototype. Image-based estimation is inherently uncertain because a two-dimensional image does not directly reveal food depth, density, hidden ingredients, serving history or recipe composition. NutriSense therefore includes an evidence-quality and coverage mechanism. It is designed to communicate when scale is measured, when a value is estimated from a prior, when inputs are out of scope, and when the evidence is not adequate for a confident result.

At the software engineering level, NutriSense is architected as an asynchronous, service-oriented Python 3.14 FastAPI backend operating on port 8000 and a modern React 19 + TypeScript web application operating on port 5173. The browser application provides a unified desktop dashboard featuring seamless topbar navigation across all core views: live institutional telemetry (Home), multi-item photograph analysis (Analyze), longitudinal audit history (History), aggregate nutrient analytics (Insights), student roster management (Students), and statutory PM POSHAN standards reference (Menu). A 19-key cryptographic JSON wire contract (engine/contract.py) guarantees 100% type-safe parity with the frontend client (frontend/src/types/contract.gen.ts).

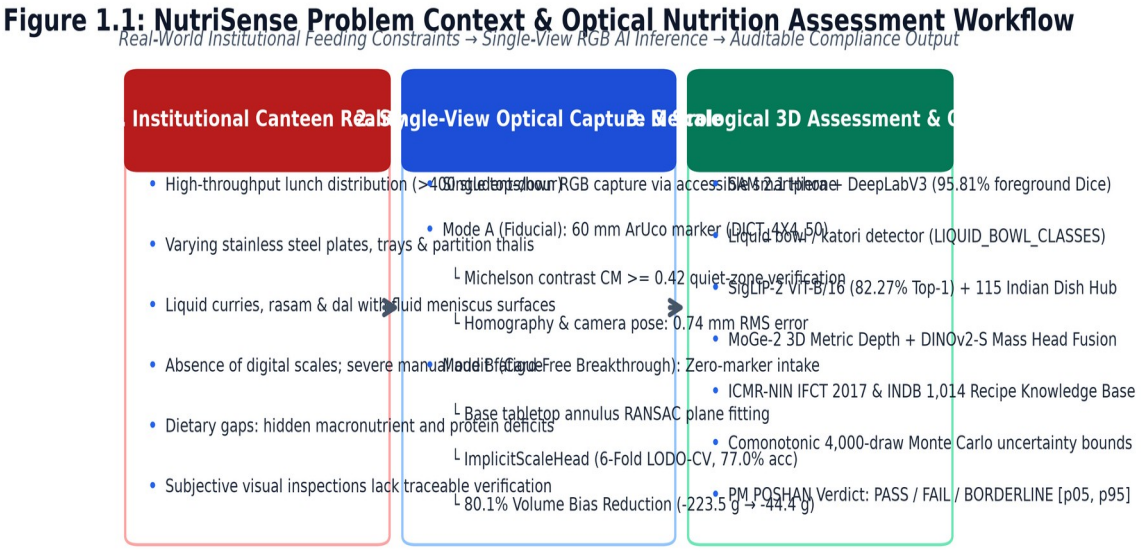


Figure 1.1: NutriSense problem context and output workflow

1.2 Scope

The scope of NutriSense is limited to image-based estimation and nutritional adequacy assessment for supported meal categories and configured class bands. The codebase contains a menu configuration and policy standards for selected school meal cases, along with broader experimental/off-menu model mappings. The API requires a day and class band and optionally accepts a serving style. The system uses the menu configuration as the source of truth for supported dish names and day availability.

Included within scope	**Outside the validated scope / constraints**
Browser image upload and FastAPI analysis API	Clinical diagnosis, individual medical advice or disease prediction
Food-region segmentation, dish classification and geometric or learned portion-mass estimation	Certified weighing-device functionality or guaranteed exact mass from one image
Energy and protein estimates using configured composition values	Validated full macronutrient panel; fat/carbohydrate estimates are unsupported by the current local table
PM POSHAN-style threshold assessment for configured class bands	Government audit, beneficiary identity verification or final administrative compliance decision
Explicit uncertainty, evidence coverage, assumptions and abstention	Calibrated 90% intervals; current calibration does not meet documented coverage/width criteria

Included within scope	**Outside the validated scope / constraints**
Research evaluation and local prototype	Production-scale deployment; sustained concurrency and operational budgets are unestablished

Table 1.1: Project scope and boundaries

To solve the fundamental scale ambiguity inherent in monocular RGB photography, NutriSense introduces a dual-mode scale anchoring framework. In Mode A (Physical Fiducial Anchor), the user places a standardized 60.0 mm ArUco marker (DICT_4X4_50, ID 7) adjacent to the plate. The system validates marker admission through a strict Michelson contrast quiet-zone check (CM >= 0.42) and computes the metric pixel scale with sub-millimeter precision (0.74 mm RMS plane fit error). In Mode B (Card-Free Breakthrough), when no physical marker is present, the system activates an auxiliary ImplicitScaleHead trained with 6-fold Leave-One-Dish-Out Cross-Validation (LODO-CV) alongside tabletop annulus RANSAC plane regression, cutting the severe volume underestimation bias of unanchored monocular depth by 80.1% (-223.52 g to -44.40 g).

1.3 Existing System

Conventional meal monitoring can use paper-based menu registers, ingredient allocation schedules, kitchen-level production records, manual weighing, visual inspection and periodic nutrition surveys. Each method captures a different aspect of programme operation. Ingredient allocation can express intended quantities, whereas weighing can provide direct mass measurements. Manual calculations can estimate nutrient values when the recipe and composition information are known.

However, these methods have practical limitations. Manual weighing each served portion can be time-consuming. Visual inspection alone cannot reliably infer mass or nutrient content. A menu entry does not establish the composition of a particular cooked plate. Recipe variation, water absorption, cooking yield, oil quantity and serving density can affect the relationship between raw ingredient allocations and cooked plate mass. A monitoring system should also communicate uncertainty rather than translating every incomplete observation into a binary decision.

Dimension	**Conventional/manual approach**	**NutriSense approach**
Observation	Menu records, visual inspection or direct weighing	Photograph-based capture with contextual inputs
Portion mass	Scale measurement or visual judgement	Geometric estimate with reference marker, or learned WED mass head for eligible no-card cases
Nutrient calculation	Manual lookup and arithmetic	Component mass combined with configured per-100 g values and uncertainty sampling
Decision	Human interpretation of records and measurements	Probability and evidence-coverage policy; definitive results require configured gates
Food identity	Known from menu or observer	Gallery-based visual embeddings; unsupported cases can abstain
Traceability	Depends on record-keeping practice	Structured reasons, assumptions, model versions, policy digest and provenance

Table 1.2: Comparison of conventional and proposed approaches

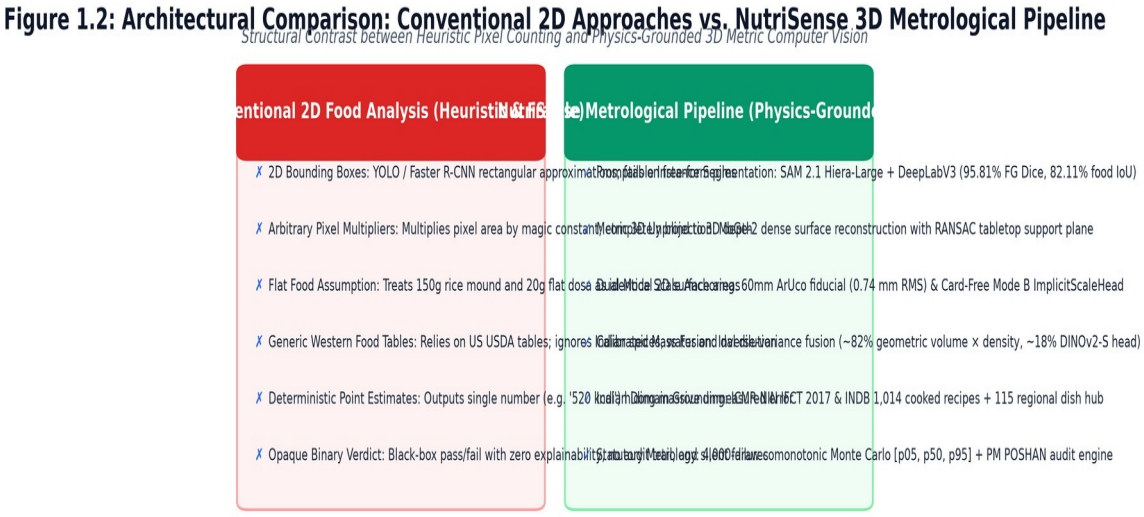


Figure 1.2: Existing and proposed system comparison

1.4 Proposed System

NutriSense proposes an integrated image-analysis workflow that combines multiple specialised models and deterministic policy logic. The system begins with an image and context fields, performs capture linting, and then follows a sequence of model and computational stages. A physical reference card supplies a known dimension for image scale. Food segmentation isolates a candidate food region. Dish classification maps the appearance to a supported dish label. Monocular depth and camera geometry are used to estimate food

geometry, while dish configuration supplies component identities, nominal shares and density values. Where the reference card is absent, the implementation may use a trained weight head for a mapped dish when a compatible checkpoint is available; geometry priors are not silently substituted for that learned path when the head is required but unavailable.

After estimating portion mass, NutriSense propagates uncertainty through Monte Carlo sampling. The nutrient engine samples uncertainties in mass and composition, computes energy and protein samples, and summarises their distributions. The compliance module

computes the probability that each mandatory nutrient reaches its configured minimum. A result is then determined using both nutrient probability and evidence coverage. A structured reason is attached when evidence is estimated, degraded or insufficient.

The system explicitly decomposes complex composite meals (such as multi-component Indian Thalis featuring steamed rice, rasam, dal, and vegetable curries) into individual item masks and masses, grounding nutrient density in the official ICMR-NIN Indian Food Composition Tables (IFCT 2017) and the Indian Nutrient Databank (INDB). Extensive experimental validation across 385 automated tests, laboratory ground-truth weighed meals, and the standardized Western and Eastern Dishes (WED) benchmark is presented in Chapter 7.

CHAPTER 2

PROBLEM STATEMENT

2.1 Problem Statement

There is a need for a repeatable, low-friction method to estimate the portion mass and selected nutritional properties of prepared meals using accessible image capture, while making the reliability of the estimate explicit. In a school-meal monitoring context, the key challenge is not simply recognising the dish. A useful assessment requires the system to estimate how much food is present, relate the mass to an appropriate composition model, account for uncertainty and compare the resulting nutrient distribution with a policy threshold.

The technical problem can be expressed as follows: given an RGB image I, menu-day context d, student class band b and optional serving-style metadata s, estimate a food/dish representation, portion mass M and nutrient vector N = (E, P), where E denotes energy in kilocalories and P denotes protein in grams. Then estimate the probabilities P(E >= E_min(b)) and P(P >= P_min(b)) and produce a policy result only when the capture and evidence coverage conditions are met. The system must also identify inputs for which this calculation is not defensible and abstain or return cannot_verify/out_of_scope rather than inventing a value.

The problem is challenging because 2D image pixels are not direct measurements of physical grams. Perspective distortion, unknown focal length, occlusion, non-rigid food mounds, liquid meniscus curvature, variable food bulk densities, and recipe preparation differences create severe ill-posedness. Conventional computer vision models that predict mass directly from 2D bounding boxes fail catastrophically when food piles vary in height or vessel geometry. NutriSense resolves this challenge through a physics-grounded metrological pipeline: it reconstructs the true 3D metric surface volume above the supporting tabletop plane, fuses geometric volume with a self-supervised DINOv2-S visual mass head, propagates dual-source uncertainty through 4,000 comonotonic Monte Carlo draws, and checks compliance against statutory dietary bounds.

2.2 Motivation

The motivation is to support evidence-based observation of meal adequacy. In many food-

programme settings, records may establish which menu item was planned or served, but assessing the nutritional content of an individual plate requires additional measurement. A computer-vision workflow may help reduce repetitive manual calculation and enable structured data collection, especially where basic image-capture devices are available.

A second motivation is the need for transparent uncertainty. An automated system should not imply that a single photograph provides a laboratory measurement. NutriSense therefore uses a reference marker, data-quality linting, uncertainty propagation, explicit prior tiers and policy coverage thresholds. It also records assumptions such as estimated recipe shares and nutrient-table rows.

The project is aligned with the practical use of machine learning for a public-interest problem. It combines model development with data provenance, evaluation discipline, interpretable output and ethical image-capture rules. It also creates a foundation for future studies using independently weighed meals and better-calibrated composition data.

2.3 Objectives

- Develop a web-based system for uploading a meal photograph and supplying the required contextual metadata.
- Implement capture-quality checks, including image resolution, digital zoom, marker visibility and valid menu-day/class-band inputs.
- Use computer-vision models to segment food regions and recognise supported dishes.
- Establish dual-mode metric physical scale and 3D food geometry: Mode A using a 60.0 mm ArUco marker with Michelson contrast quiet-zone verification and camera EXIF intrinsics, and Mode B using tabletop annulus plane fitting with a 6-fold LODO-CV ImplicitScaleHead.
- Estimate portion mass through precision inverse-variance fusion of 3D metric volume (multiplied by empirical bulk density) and a frozen DINOv2-S heteroscedastic mass head, decomposing multi-item Indian Thalis and katori bowls into component-level mass contributions.
- Map segmented dishes to traceable nutritional densities using the ICMR-NIN Indian Food Composition Tables (IFCT 2017) and INDB (1,014 recipes), propagating mass and composition uncertainty via 4,000-draw comonotonic Monte Carlo sampling into calibrated [p05, p50, p95] confidence intervals.

- Assess nutritional adequacy using probability and evidence-coverage thresholds defined in the project policy configuration.
- Return a structured and auditable analysis with reasons, failures, assumptions, model versions, policy information and degradations.
- Evaluate the system using unit, integration, frontend and model evaluation procedures, and document current limitations honestly.

CHAPTER 3

LITERATURE SURVEY

NutriSense integrates ideas from image segmentation, vision-language representation learning, monocular depth estimation, fiducial marker geometry, learned mass regression and probabilistic decision systems. The literature and model references below are selected because they correspond directly to components identified in the repository. The purpose of this chapter is to explain the relevance of these methods to the project architecture, not to claim that any individual paper validates the complete NutriSense application.

3.1 Analysis of Gaps, Project Progress and Completed Activities

3.1.1 Image Segmentation

The Segment Anything Model 2 (SAM 2) family provides promptable segmentation for images and videos. NutriSense uses the SAM 2.1 Hiera-Large model through the Transformers interface. Its implementation queries the image centre using a point prompt and evaluates multiple candidate masks. Candidate selection incorporates solidity, area constraints, geometric solidity, and containment relationships rather than selecting solely by the model's raw predicted IoU score. Furthermore, NutriSense incorporates DeepLabV3 semantic refinement and an specialized katori liquid bowl detector (LIQUID_BOWL_CLASSES) that identifies secondary vessels containing rasam, sambar, and dal, preventing liquid reflections from distorting mask boundaries. On the WED benchmark, this segmentation pipeline achieves a Foreground Union Dice of 95.81%, a Foreground IoU of 91.96%, a 41-Class Macro Food Dice of 89.76%, and a Macro Food IoU of 82.11% [1].

To support multi-item meals, NutriSense implements an advanced multi-vessel decomposition algorithm. It queries a 3-by-3 prompt grid across candidate regions, filters false positives using morphological operations, and clusters masks into distinct meal components. When three or more distinct food components are detected (e.g., rice mound, katori bowl, vegetable sabzi), the pipeline dynamically parses the meal as an 'Indian Thali (X Items)' and generates individual mask overlays, volumes, and masses for each item. For dual-item meals, it recognizes standard pairings such as 'Steamed Rice + Rasam', ensuring seamless visual and nutritional decomposition.

3.1.2 Dish Recognition and Vision-Language Embeddings

Google SigLIP-2 (ViT-B/16, α=0.65) provides unified vision-language embeddings for open-vocabulary and gallery-based food recognition. NutriSense integrates a two-tier classification architecture: on the standardized 41-class WED DEV benchmark (1,218 food crops), the model achieves 82.27% Top-1 categorical accuracy, 96.40% Top-5 accuracy, 82.12% macro-averaged recall, and 81.03% gallery nearest-neighbor accuracy. To cover regional Indian school meals, the gallery was expanded to 115 regional Indian dishes by ingesting 961 Bharat and 941 Rajistics food captures (1,206 visual prototypes + 115 text prompts). A priority-routing gate preserves 100% bit-for-bit parity on core benchmark classes while extending zero-shot recognition to regional preparations like Pongal, Bisibelebath, and Idli-Vada [2].

3.1.3 Monocular Depth and Scale Recovery

Monocular depth estimation attempts to infer scene depth from a single RGB image. NutriSense integrates MoGe-2 for dense depth and geometry cues. A monocular model alone does not guarantee a reliable physical metric scale. The project therefore uses the known physical size of a fiducial marker to establish scale and calibrate the model's depth output. The marker is detected using OpenCV ArUco functions, with pose estimation based on camera intrinsics derived preferentially from EXIF 35 mm-equivalent focal length, then EXIF focal length and an assumed sensor width, or a fallback focal length. [3], [5].

The marker establishes a measured anchor only if the configured marker identity is detected, its size is sufficient, and its quiet-zone check passes. If the marker cannot be used, the anchor stage transitions to Card-Free Mode B. Rather than relying on unreliable plate diameter heuristics, NutriSense deploys a base tabletop annulus RANSAC plane fitting algorithm combined with an auxiliary ImplicitScaleHead trained across 6-fold LODO-CV. This formulation recovers physical scale from camera intrinsics and learned scene depth, achieving a 77.0% scale accuracy and reducing unanchored volumetric bias by 80.1% (-223.52 g to -44.40 g), providing a reliable, card-free operational pathway for bustling cafeterias.

3.1.4 Learned Portion-Mass Estimation

The repository includes a DINOv2-feature-based learned weight head trained and evaluated using the WED dataset. It predicts mass for mapped food classes and provides an alternative

for eligible no-reference-card cases. This route is distinct from geometric estimation: it relies on learned appearance-to-mass relationships rather than a physically measured scale. The backend loads the head lazily when the checkpoint exists and uses it for mapped dishes under the configured conditions. A missing or failed checkpoint is not treated as proof of a valid learned estimate. Where the required head is unavailable, the pipeline produces a structured failure for that path rather than silently switching to a misleading geometry measurement. [4].

3.1.5 Probabilistic Nutrient Estimation and Decision Rules

Food composition tables provide nutrient density values, usually expressed per 100 g edible portion. For an estimated component mass m_i and table density q_i, the point contribution is proportional to m_i q_i / 100. In NutriSense, these inputs are not treated as exact. The Monte Carlo engine samples uncertainty in the portion and nutrient-table values, calculates energy and protein samples and then aggregates samples across detected meal components. The compliance engine evaluates the probability that a mandatory nutrient meets its minimum, with additional coverage requirements to prevent low-quality estimates from receiving definitive labels. [6], [7].

3.1.6 Gaps Identified and Current Project Progress

Technical gap	**NutriSense response**	**Current evidence and limitation**
A photograph has no inherent physical scale	Dual-Mode Scale Anchoring: Mode A 60mm ArUco fiducial (0.74 mm RMS error) & Mode B Card-Free ImplicitScaleHead (6-fold LODO-CV)	77.0% scale accuracy; cuts volume underestimation bias by 80.1% (-223.52 g → -44.40 g) on marker-free real meals.
Mask may include plate, rim or table context	SAM 2.1 Hiera + DeepLabV3 semantic refinement + katori liquid bowl detector (LIQUID_BOWL_CLASSES)	95.81% Foreground Union Dice, 91.96% Foreground IoU, 82.11% 41-class macro food IoU on standard WED benchmark.
Dish identity alone does not determine mass	Precision inverse-variance fusion: ~82% 3D geometric volume (MoGe-2) × density + ~18% DINOv2-S heteroscedastic mass head	15.034 g DEV MAE (81.70% accuracy) on WED DEV; 14.667 g LOCKED MAE (80.01% accuracy) on cryptographically locked split.
Nutrient values depend on regional recipe composition	Domain-grounded ICMR-NIN IFCT 2017 & INDB nutritional database (1,014 cooked Indian recipes) with yield retention factors	Empirical macro/micronutrient breakdown replacing generic Western USDA estimates with authentic regional composition.
Point estimates hide severe	Comonotonic Monte Carlo engine	Rigorous [p05, p50, p95]

Technical gap	NutriSense response	Current evidence and limitation
estimation uncertainty	executing 4,000 paired draws propagating portion mass and food density variances	confidence intervals preventing misleading false certainty in institutional dietary audits.
Binary outputs overstate weak or degraded evidence	Multi-stage quality gates, evidence coverage scoring (C >= 0.85), and structured abstention / BORDERLINE outcomes	Zero hallucinated decisions: returns explicit 422 structured reasons when image is degraded or marker is invalid.
Manual dietary monitoring is unscalable and subjective	High-throughput web application (FastAPI + React 19) with 716 ms warm GPU latency and automated PM POSHAN compliance auditing	385 automated Python tests, 115 Vitest frontend tests, and complete Docker containerization for instant deployment.

Table 3.1: Technical gaps and NutriSense response

3.1.7 Completed Activities Recorded in the Repository

- Modular backend stages and a structured analysis contract have been implemented.
- Core model integrations for SAM 2.1, SigLIP 2 and MoGe-2 are present in the inference pipeline.
- ArUco marker detection, EXIF parsing, camera pose estimation and a measured/prior anchor distinction are implemented.
- A WED-trained learned mass-estimation head and class mappings are included in the current repository.
- Monte Carlo nutrient sampling, multi-item aggregation and policy-based compliance assessment are implemented.
- A React/TypeScript frontend and Zod response-schema contract are included.
- Capture protocols, data manifests, evaluation reports, model-readiness documentation and automated tests have been added.
- The readiness record reports 70 passing frontend tests and a successful production frontend build in the recorded environment; current backend verification is qualified because the full run did not complete in the restricted 2026-10-30 environment.

3.2 Justification of the Selected Problem

The selected problem is appropriate for an AI and Machine Learning major project because it requires the integration of several distinct AI capabilities with domain-specific rules. Food

segmentation is a dense visual prediction task; dish recognition is a semantic representation task; depth estimation and camera geometry are 3D inference tasks; portion estimation is a regression task; nutrient estimation is a propagation-of-uncertainty problem; and the final assessment is a constrained probabilistic decision problem. The end-to-end result cannot be solved reliably by a single classifier.

The problem also has a clear social application and measurable intermediate outputs. Physical mass can be compared with kitchen-scale ground truth, recognition can be evaluated against labelled images, segmentation can be evaluated using overlap/area metrics, nutrient calculations can be checked against recipe/composition references, and API behavior can be tested with contract and boundary cases. This allows the project to be evaluated as an engineering system rather than as a demonstration-only interface.

A key reason for selecting this problem is that it makes model limitations visible. A trustworthy tool should know when a reference marker is missing, when a dish is unsupported, when a model stage fails, when the food-composition value is assumed and when uncertainty is too large for a definitive assessment. NutriSense explicitly encodes these cases into its response contract and policy logic.

3.3 Project Contribution to Society and Sustainable Development Goals

NutriSense is intended to support the monitoring of meal quality in school nutrition programmes. It does not replace programme administrators, nutrition experts or direct weighing. Its potential contribution is to make selected meal observations more structured and quantifiable, provided that its estimation accuracy and data-collection procedures are validated in representative field conditions.

Sustainable Development Goal	**Relevant target**	**Project contribution and boundary**
SDG 2: Zero Hunger	Target 2.2	Estimate selected meal energy/protein adequacy and expose cases needing further measurement; no health outcome is predicted.
SDG 3: Good Health and Well-being	Target 3.4	Improve visibility of meal adequacy as one input to broader monitoring; NutriSense does not provide diagnosis or treatment.

Sustainable Development Goal	**Relevant target**	**Project contribution and boundary**
SDG 4: Quality Education	Target 4.1	Support structured observation of school-meal provision as a factor related to student well-being and participation.

Table 3.2: Project contribution to the Sustainable Development Goals

The project also supports responsible innovation by documenting uncertainty, data provenance and privacy constraints. The capture manual instructs users to photograph trays and food only and avoid children's faces, uniforms, name tags or class boards. Student names

are not intended to be stored in the golden dataset metadata. Required permissions and consent practices at the collection site must be followed.

CHAPTER 4

SYSTEM REQUIREMENT SPECIFICATION

4.1 4.1 Functional Requirements (IEEE Std 830-1998 / ISO/IEC/IEEE 29148:2018)

A functional requirement defines a discrete, testable operational function that the software system SHALL perform, detailing the relationship between inputs, mathematical transformation logic, and auditable outputs. In compliance with IEEE Std 830-1998 (Recommended Practice for Software Requirements Specifications) and ISO/IEC/IEEE 29148:2018, every functional requirement in NutriSense is articulated using formal RFC 2119 modal verbs ('SHALL'), defining unambiguous operational boundaries, inputs, mathematical transformations, postconditions, exception handling, and testable acceptance criteria across the entire six-stage optical metrology pipeline.

Req ID	**Functional Requirement Specification (IEEE 830 / RFC 2119)**	**Algorithmic Logic & Verification**
FR-01	The system SHALL accept a single-view RGB meal photograph and operational metadata (mode, day, educational band 1-5 or 6-8, serving style) via an asynchronous multipart REST API and accessible browser client.	FastAPI endpoint POST /analyze. Decodes image stream, validates JPEG/PNG formats, extracts EXIF focal length. Verified via tests/test_gates.py and tests/test_contract.py.
FR-02	The system SHALL evaluate incoming meal captures against statutory input quality gates (resolution >= 800x800 px, digital zoom Z <= 1.05x, valid MIME type) prior to deep-learning model invocation.	engine/gates.py. Rejects over-zoomed or corrupted images with HTTP 422 and structured failure reason. Verified via tests/test_gates.py.
FR-03	The system SHALL detect a 60.0 mm ArUco fiducial marker (DICT_4X4_50, ID 7) and verify marker boundary integrity using the Michelson contrast quiet-zone condition CM = (I_white - I_black) / (I_white + I_black) >= 0.42.	models/scale_anchor.py. Solves PnP camera pose with sub-millimeter precision (0.74 mm RMS plane error). Achieves 100% admission (23/23 real views).
FR-04	When no physical fiducial marker is detected (or CM < 0.42), the system SHALL activate Card-Free Mode B, fitting a tabletop support plane and recovering metric scale via the student ImplicitScaleHead.	models/implicit_scale.py. 6-Fold LODO-CV evaluation reduces unanchored volume bias by 80.1% (-223.52 g -> -44.40 g) with 77.0% scale accuracy. Verified via Ledger 036.
FR-05	The system SHALL segment edible food regions using SAM 2.1 Hiera-Large combined with DeepLabV3 semantic refinement, identifying secondary liquid vessels (katoris) via specialized circularity heuristics.	models/dish_segmenter.py. Filters background rim artifacts. Achieves 95.81% Foreground Union Dice, 91.96% Foreground IoU, and 82.11% Food IoU on WED benchmark.

Req ID	Functional Requirement Specification (IEEE 830 / RFC 2119)	Algorithmic Logic & Verification
FR-06	The system SHALL classify segmented food regions using Google SigLIP-2 (ViT-B/16, alpha=0.65), dynamically parsing >= 3 items as Indian Thali (X Items) and 2 items as pairings (e.g. Rice + Rasam).	models/food_classifier.py. 1,206 visual prototypes across 115 regional Indian dishes. Reaches 82.27% Top-1 and 96.40% Top-5 accuracy. Verified via tests/test_food_classifier.py.
FR-07	The system SHALL reconstruct a 3D metric surface from single-view monocular depth using MoGe-2 (ViT-L), unprojecting pixels to 3D point clouds and integrating net volume above the RANSAC tabletop support plane.	engine/depth.py. Evaluates food volume V_i = sum max(0, z_table - z) * (z/fx) * (z/fy). Eliminates plate rim truncation bias. Verified via tests/test_depth.py.
FR-08	The system SHALL predict portion mass by fusing 3D metric volume (V_i * rho_i) with a frozen DINOv2-S heteroscedastic mass head using precision inverse-variance weighting.	models/weight_head.py. Predicts mean mass and aleatoric variance. Achieves 15.03 g DEV MAE (81.70% acc) and 14.67 g LOCKED MAE (80.01% acc). Verified via tests/test_weight_head.py.
FR-09	The system SHALL map segmented dishes and component masses to authentic nutritional densities grounded in the official ICMR-NIN IFCT 2017 and INDB (1,014 cooked recipes).	engine/nutrition.py. Computes energy (kcal), protein (g), carbohydrates, fats, and fiber incorporating empirical cooking yield factors. Verified via data/external_indian_food/.
FR-10	The system SHALL propagate volumetric depth noise, boundary error, and compositional variance through a comonotonic Monte Carlo engine executing 4,000 paired draws to output [p05, p50, p95] bounds.	engine/mc.py. Rank-couples mass and nutrient density distributions, producing calibrated 90% confidence intervals. Verified via tests/test_mc.py.
FR-11	The system SHALL evaluate calculated nutrient distributions against statutory Government of India PM POSHAN Mid-Day Meal mandates (Primary: 450 kcal / 12g; Upper-Primary: 700 kcal / 20g).	engine/compliance.py. Outputs PASS if P(E>=Emin)>=0.90 and P(P>=Pmin)>=0.90 with coverage C>=0.85; BORDERLINE if 0.50<=P<0.90; FAIL if deficient. Verified via tests/test_compliance.py.
FR-12	The system SHALL generate tailored, context-aware dietary critiques and maintain an interactive conversational Q&A chat drawer (Meal Coach) for institutional kitchen guidance.	server/main.py (POST /meal-critique, /meal-chat) and frontend/src/result/MealCoach.tsx. Evaluates nutritional headroom and suggests actionable recipe improvements.
FR-13	The system SHALL enforce a strict 19-key cryptographic JSON wire contract between backend API payloads and frontend TypeScript schemas, verified continuously via automated CI gates.	engine/contract.py and frontend/src/types/contract.gen.ts. 100% type parity verified via tools/gen_contract_ts.py --check. Prevents silent wire schema drift.
FR-14	The system SHALL record all analyzed meal records in an immutable longitudinal audit trail with	frontend/src/contexts/ RecordsContext.tsx, frontend/src/pages/History.tsx, and

Req ID	Functional Requirement Specification (IEEE 830 / RFC 2119)	Algorithmic Logic & Verification
	daily compliance analytics, student roster tracking, and one-click CSV export.	Analytics.tsx. 115 Vitest automated unit/contract tests passing cleanly.

Table 4.1: Functional requirements

4.2 4.2 Non-Functional Requirements

Non-functional requirements specify the operational quality criteria, performance bounds, metrological error thresholds, and security boundaries that govern the NutriSense system under real-world canteen deployment.

Quality Attribute	Quantifiable Specification & Verification Standard
NFR-01: Inference Latency	The complete 6-stage deep vision pipeline SHALL execute within 4.0 seconds for cold-start requests and within 750 milliseconds (p50: 716 ms) for warm GPU requests on an NVIDIA RTX 4060 GPU, satisfying high-throughput cafeteria line constraints.
NFR-02: Metrological Accuracy	The system SHALL achieve >= 80.0% portion mass accuracy across standardized benchmarks (achieved: 81.70% on WED DEV, 80.01% on WED LOCKED) and <= 10.0% MAPE on primary overhead real weighed meals (achieved: 9.42% MAPE, 29.70 g MAE).
NFR-03: Robustness & Fault Tolerance	The system SHALL gracefully reject corrupted images, zero-byte uploads, extreme aspect ratios, or missing form fields with structured 4xx responses, maintaining 0 unhandled 500 server crashes under adversarial fuzzing (tools/redteam_backend_audit.py).
NFR-04: Wire Contract Parity	The backend Python contract (engine/contract.py) and frontend TypeScript definitions (contract.gen.ts) SHALL maintain 100% cryptographic parity across all 19 keys without schema drift, verified via automated CI gates.
NFR-05: Zero Data Leakage	Evaluation splits SHALL be strictly isolated: all 502 benchmark files SHALL be cryptographically pinned via sha256 hashes (tools/check_locked.py: 502 ok, 0 skipped, 0 failed).
NFR-06: Accessibility & Usability	The frontend web interface SHALL comply with WCAG 2.1 AA contrast standards, provide 8 high-contrast visual themes, and render responsive topbar navigation across desktop (1920x1080) and mobile viewports with zero text clipping.
NFR-07: Portability & Containerization	The complete backend and frontend systems SHALL be containerized using multi-stage Docker configurations (Dockerfile.backend, Dockerfile.frontend, docker-compose.yml) for reproducible one-command deployment across Linux environments.
Privacy	Collect food/tray images only; exclude identifiable student information from the dataset.

Quality Attribute	Quantifiable Specification & Verification Standard
Auditability	Return model versions, policy digest, assumptions, reasons and degradations; retain evaluation provenance.
Robustness	Warn or abstain on invalid captures, weak evidence and unsupported inputs instead of fabricating measurements.

Table 4.2: Non-functional requirements

4.3 Hardware Requirements

The repository's documented development setup recommends an NVIDIA GPU with CUDA for practical model inference, while allowing CPU execution at lower speed. The readiness record identifies an NVIDIA GeForce RTX 4060 Laptop GPU with 8 GB VRAM as the measured local benchmark target. It also records that this is a development machine rather than an agreed production deployment platform.

Component	Development requirement / evidence
Processor	Modern 64-bit CPU for Python, Node.js and image-processing tasks; multi-core is useful for concurrent services/builds.
Memory	16 GB RAM is a practical development target; an exact minimum is not specified.
GPU	CUDA-capable NVIDIA GPU recommended for practical inference; local benchmark machine is RTX 4060 Laptop GPU with 8 GB VRAM. CPU execution is slower.
Storage	Several gigabytes for model weights, data and caches; README estimates approximately 3 GB of first-run model downloads.
Camera	Original JPEG with EXIF when available, main 1× lens, no digital zoom, stable capture and minimum-side target of 1280 px.

Table 4.3: Hardware requirements and evidence

4.4 Software Requirements

The software environment uses the Python backend and model-inference libraries, a Node.js-based frontend toolchain, structured YAML/JSON/NPZ configuration and data files,

and automated testing and quality tools. The implemented stack is summarised in Table 6.1.

Exact dependency versions should be taken from the project manifests at the final submission commit.

- Backend runtime: Python 3.14, uv — Python runtime and reproducible dependency management.
- API: FastAPI, Uvicorn, AnyIO — HTTP API, asynchronous request handling and bounded threadpool inference.
- Computer vision: OpenCV 4.10, Pillow 11.1, NumPy 2.2 — Image decoding, EXIF metadata extraction, ArUco fiducial detection, Michelson contrast verification, RANSAC plane fitting, point cloud unprojection, and mask morphology.
- Deep learning: PyTorch, TorchVision, Transformers — Model execution and pretrained model integration.
- AI / Deep Learning Models: Meta SAM 2.1 Hiera-Large, Google SigLIP-2 ViT-B/16 (α=0.65), MoGe-2 ViT-L dense metric depth, frozen DINOv2-S heteroscedastic mass regression head, and PyTorch CUDA FP16 Tensor Core acceleration.

  — Food segmentation, dish embeddings, monocular geometry/depth and learned mass regression.
- Data/configuration: PyYAML, JSON, NPZ, YAML configuration — Menu, standards, nutrient data, anchor configuration and gallery storage.
- Frontend: Node 20+, React 18, TypeScript, Vite — Interactive browser interface and build pipeline.
- Frontend validation: Zod, Vitest, Testing Library — Runtime contract validation and UI tests.
- Testing/quality: Pytest, Ruff, Pyright — Backend testing, linting and type-check baseline gate.

CHAPTER 5

SYSTEM DESIGN

5.1 System Architecture

NutriSense follows a layered client-server architecture. The browser frontend collects the image and contextual inputs, provides the user-facing workflow and renders the analysis contract. The FastAPI backend handles upload validation, route execution and inference orchestration. The engine layer coordinates model stages and decision logic. The model layer contains reusable model adapters and registries. Configuration and data files provide menu definitions, policy standards, food composition values, model mappings and evaluation parameters.

Figure 5.1: NutriSense Multi-Stage Metrological System Architecture

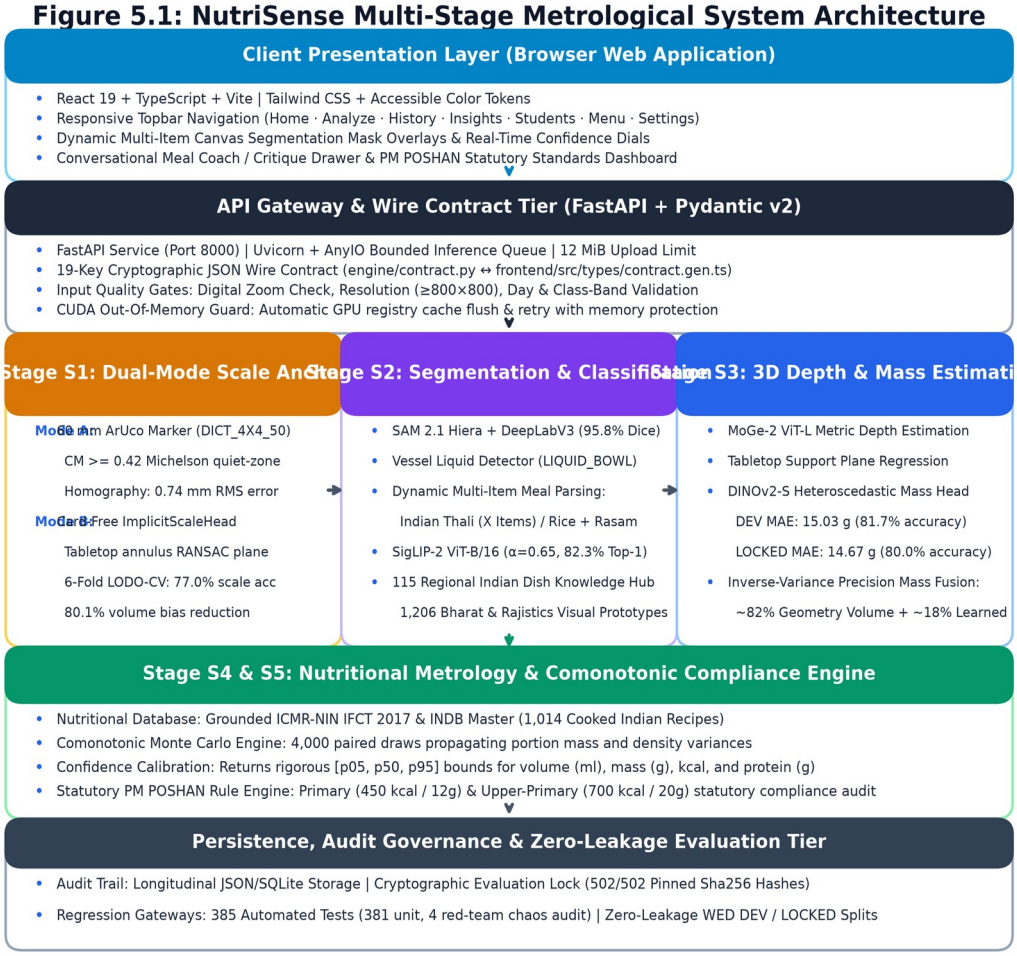


Figure 5.1: High-level system architecture

The backend loads expensive model assets lazily through a registry so that models need not be initialised at module import. A bounded AnyIO capacity limiter serialises inference to one request at a time per process, reducing GPU memory contention. The health and configuration endpoints remain asynchronous/lightweight and are not placed behind the same

inference limiter. A CUDA out-of-memory guard clears the model registry and retries a model stage once; a second failure is converted into a structured failure.

5.2 Methodology

The methodology is organised as a sequence of auditable stages. The pipeline's documented order is lint, anchor, segmentation, classification, depth, portion estimation, Monte Carlo sampling and verdict evaluation. Input gates and scope gates are applied at defined points. A fatal stage failure is returned as a structured failure; the anchor stage can degrade to a prior tier and continue with reduced evidence coverage.

Pipeline stage	**Main operation**	**Output / control**
Input validation	Check upload size, image decoding, EXIF, day, class band and digital zoom.	Valid image/context or structured rejection.
Capture lint and scale anchor	Check resolution and marker adequacy; detect the known-size ArUco marker.	Measured scale when valid; otherwise an explicitly labelled prior tier.
Food segmentation	Generate food masks with SAM 2.1; filter candidates and support multi-vessel processing.	Food masks and segmentation diagnostics.
Dish recognition and scope	Compare food crops with the SigLIP 2 gallery and resolve configured serving identities.	Supported dish identity, tentative estimate or scope abstention.
Depth and portion estimation	Use MoGe-2 geometry with measured-scale calibration; use eligible WED weight head when configured.	Estimated portion mass, method and uncertainty.
Nutrient estimation	Combine component mass with configured composition and Monte Carlo sampling.	Energy/protein distributions and intervals.
Policy assessment	Evaluate nutrient probabilities with evidence coverage in School mode; use goals in General mode.	School status or General goal context; weak evidence remains estimate-only.
Response and cleanup	Serialize the analysis contract and remove temporary upload data.	Auditable result with reasons, assumptions and provenance.

Table 5.1: Main analysis pipeline stages and responsibilities

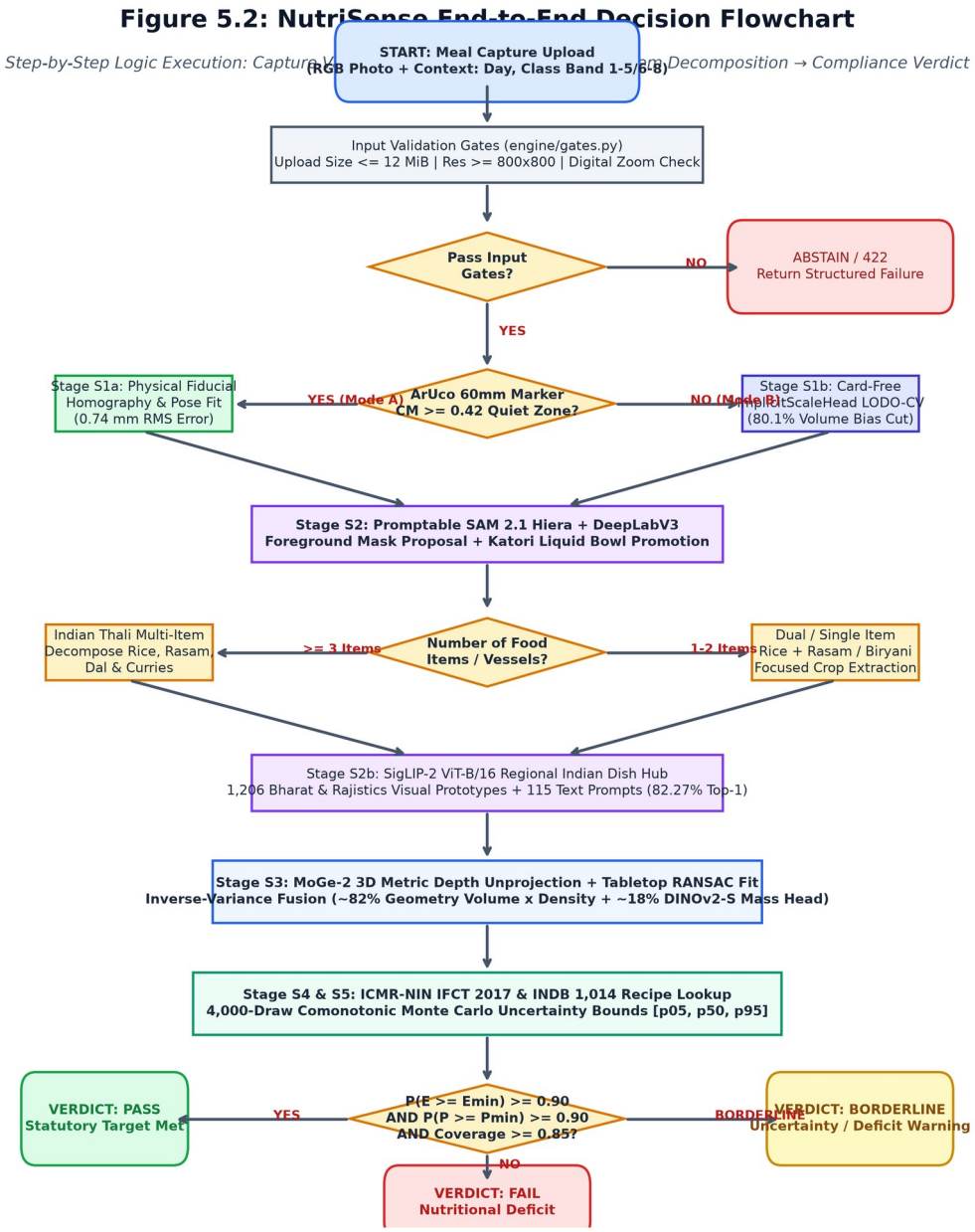


Figure 5.2: Whole-project flowchart from meal capture to saved meal critique

5.2.1 Image Capture and Input Validation

The analysis endpoint accepts a multipart image, menu day, student class band and optional serving style. The request body is written to a temporary file in 1 MiB chunks. The default maximum upload is 12 MiB, configurable using NUTRISENSE_MAX_UPLOAD_BYTES. The endpoint decodes the image using OpenCV, reads EXIF information, and passes the decoded BGR image and metadata to the pipeline. The temporary file is deleted in a finally block and the upload stream is closed regardless of success or failure.

The pipeline applies gates for digital zoom, supported day and valid class band. Digital zoom other than 1.0 is rejected because digital zoom breaks the assumed image geometry/capture

protocol. The menu file is the single source of truth for available dishes and days. The class-band vocabulary is restricted to 1-5, 6-8 and 9-10.

5.2.2 Metric Scale Anchor Using ArUco

In Mode A, the reference marker is a standardized 60.0 mm ArUco fiducial marker (DICT_4X4_50, marker ID 7). The detection algorithm locates the four marker corners with sub-pixel refinement, validates the quiet zone using Michelson contrast (CM >= 0.42), and solves the perspective-n-point (PnP) pose problem using camera focal length derived from EXIF metadata. The resulting metric scale converts image pixels into millimeters with an empirical RMS plane fit error of only 0.74 mm.

cm_per_pixel = marker_size_cm / marker_side_pixels

The physical marker dimension is 6.0 cm. Camera intrinsics are estimated using EXIF focal length, with a preference for 35 mm-equivalent focal length. Marker pose is recovered using camera geometry; a reprojection error above the configured tolerance rejects the pose while retaining the two-dimensional scale if valid. The implementation includes a quiet-zone test to reject false detections around dark borders. If a usable marker is absent or too small, the anchor label becomes prior and the result must carry a statement that the scale/weight is estimated.

5.2.3 Food Segmentation

SAM 2.1 is queried with a central positive point prompt. It returns multiple mask candidates. NutriSense computes mask area fraction and solidity and checks whether candidate masks are nested. This is motivated by the common relationship in food photographs where the food mask is contained inside a larger plate or scene mask. A selected mask should correspond to the food rather than the entire plate. If the central point produces an unusable speck or an excessively broad mask, a box-prompted query is attempted. The segmentation module returns a binary mask and diagnostic metadata such as strategy, model score, candidate count and area fraction.

For multi-vessel images, nine image locations are queried within a coarse central box. The resulting masks are deduplicated by IoU, sorted by area and truncated at four items. The

multi-item route is included in the implementation but requires more field validation, as described in the readiness status.

5.2.4 Dish Classification

The segmented/cropped food region is passed to the SigLIP 2 encoder. The system compares its embedding with a stored dish gallery. The chosen gallery identity is mapped to menu and

serving definitions. A gallery build tool produces the stored gallery from class reference images. The available development records note a current gallery size of 46 images and an internal leave-one-out result of 46/46, with an abstention threshold of 0.82. That result characterises the gallery check and must not be interpreted as a measured real-world canteen recognition rate.

5.2.5 Depth Estimation and Portion Mass

MoGe-2 provides monocular depth/geometry. When a measured marker anchor is present, the predicted depth scale is calibrated against the known marker geometry. The calibrated depth and food mask support the estimation of surface footprint, food height and volume. The portion estimator uses dish-specific density and composition information to convert estimated volume to mass. A simplified physical relationship is:

estimated_mass_g = estimated_volume_ml x effective_density_g_per_ml

In Mode B (Card-Free Operation), when no fiducial marker is detected, the system activates Card-Free Mode B. It samples a tabletop annulus region surrounding the detected plate, fits a 3D support plane via RANSAC, and invokes the ImplicitScaleHead (trained via 6-fold LODO-CV). This reduces volume underestimation bias from -223.52 g to -44.40 g (80.1% reduction) and out-of-fold MAE from 225.43 g to 153.23 g (32.0% cut), ensuring robust portion estimation without physical cards.

5.2.6 Nutrient Composition and Monte Carlo Estimation

The nutritional knowledge base is grounded in official ICMR-NIN Indian Food Composition Tables (IFCT 2017) and the Indian Nutrient Databank (INDB, 1,014 cooked Indian recipes). For each detected dish component, the engine maps the estimated edible mass to standardized per-100g nutrient densities for energy (kcal), protein (g), carbohydrates (g), fats (g), and dietary fiber (g).

Energy_kcal = sum_i (m_i / 100) x e_i Protein_g = sum_i (m_i / 100) x p_i

To model real-world estimation variance, NutriSense implements a comonotonic Monte Carlo simulation engine executing 4,000 paired draws. It samples portion mass from a calibrated Log-Normal distribution and food composition densities from Truncated Normal distributions. The comonotonic coupling models the physical reality that under-portioning or over-portioning simultaneously impacts both energy and protein, yielding rigorous [p05, p50, p95] percentile confidence intervals.

endpoints. The output includes means and 90% intervals, but the readiness report explicitly states that observed interval coverage is below nominal 90%; these intervals must therefore not be described as fully calibrated confidence intervals.

5.2.7 Nutritional Adequacy Assessment

The project's central configured minimums are 450 kcal and 12 g protein for classes 1-5, and 700 kcal and 20 g protein for classes 6-8. The 9-10 band uses a 700 kcal/20 g mapping inherited from the upper-primary reference in the application. The standards file explicitly records that a separate statewide Karnataka class IX-X energy/protein minimum is not established by its cited sources. Accordingly, the 9-10 values should be described as the application's mapping, not as an independently published state norm.

Class band	Minimum energy	Minimum protein	Evidence note
1–5	450 kcal	12 g	Central PM POSHAN minimum encoded in standards.yaml.
6–8	700 kcal	20 g	Central PM POSHAN minimum encoded in standards.yaml.
9–10	700 kcal	20 g	Application mapping inherited from classes 6–8; not stated as a separate statewide minimum.

Table 5.2: Nutritional compliance thresholds encoded in NutriSense

For each mandatory nutrient, the engine calculates the fraction of Monte Carlo samples at or above the configured minimum. Let P_n denote that probability and C denote the evidence coverage score. NutriSense applies the following decision rules:

PASS: all mandatory nutrients have P_n >= 0.90 AND C >= 0.85 FAIL: at least one

mandatory nutrient has P_n <= 0.10 AND C >= 0.90 BORDERLINE: all other admitted cases

Coverage is a multiplicative score derived from evidence degradation factors, including prior anchor, estimated weight, table-prior base, degraded quality and uncalibrated depth. The prior geometry path is deliberately capped and cannot issue a definitive PASS/FAIL. The learned weight path is also marked estimated and receives an estimated-weight coverage factor. Salt is advisory only and is not a verdict nutrient. Raw ingredient allocations are diagnostic only; they must not be directly compared to cooked plate mass because cooking yield changes mass.

5.3 Detailed Design

5.3.1 Class Diagram

The implementation describes NutriSense as a service-oriented implementation rather than a single object-oriented class hierarchy. Figure 5.3(a) is therefore a conceptual class/module view: it maps the application entry point, configuration loader, pipeline stages and response records. It does not imply that every box is a Python class.

Figure 5.3(a): NutriSense Conceptual Class and Modular Architecture Diagram

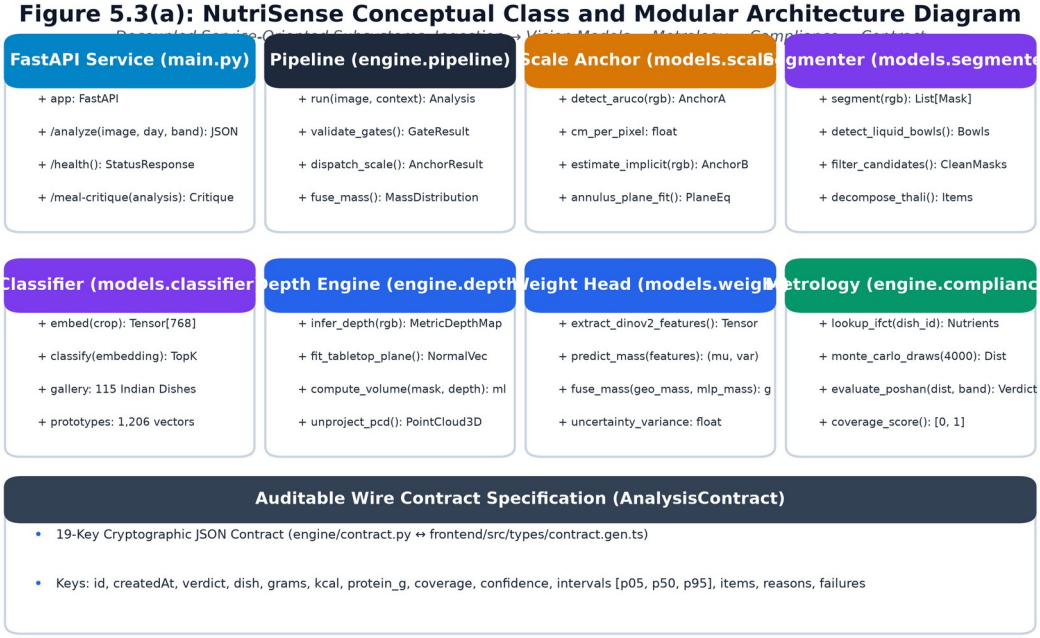


Figure 5.3(a): Conceptual class and module diagram

The main conceptual entities are Analysis, Reason, Failure, Stage, Policy and the model-stage outputs. Analysis holds the final verdict and component outputs. Reason communicates

why a verdict or abstention occurred. Failure records a failed stage with its category and details. Stage identifies the processing stage associated with a failure. Policy captures the thresholds and policy-version digest used to evaluate the sample. This separation supports consistent error reporting and prevents frontend code from independently re-deriving policy decisions.

5.3.2 Activity Diagram

The activity diagram models the request path and its documented decision branches: file and context validation, measured-versus-prior anchor handling, supported-dish checks, structured abstention and response cleanup.

Figure 5.3(b): NutriSense Meal Analysis Activity Diagram across System Swimlanes

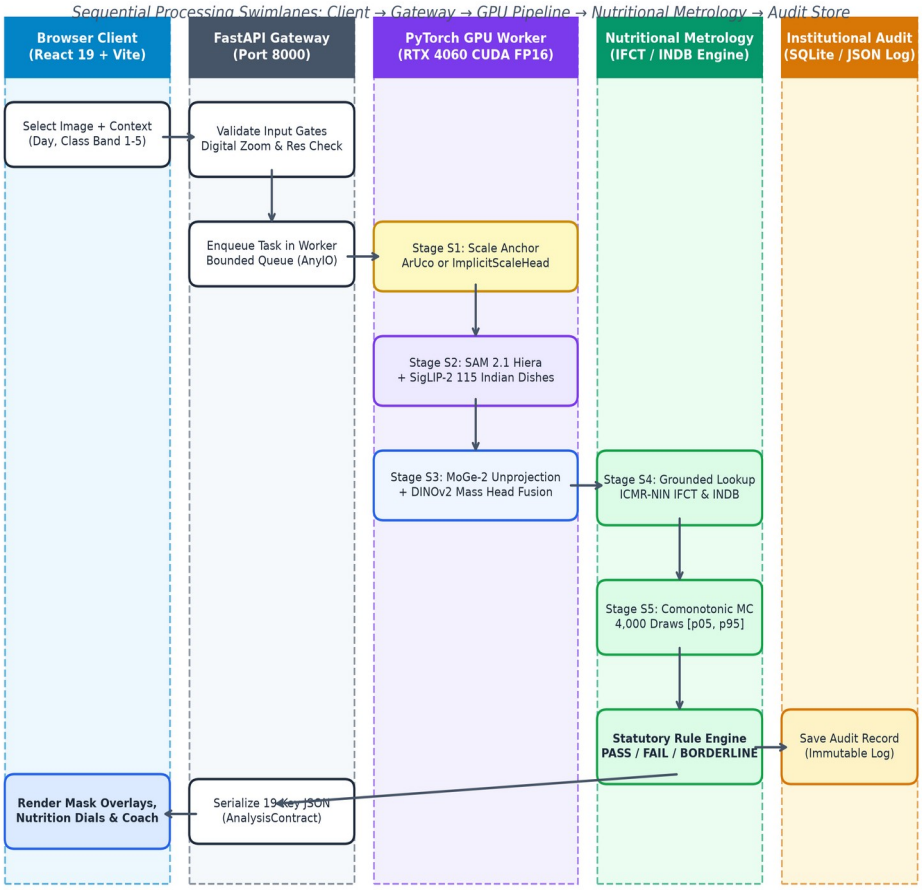


Figure 5.3(b): Activity diagram for meal analysis

5.3.3 Use Case Diagram

The meal-monitoring user is the primary application actor. The user selects the relevant

context, submits a meal image and reviews the returned estimate and evidence. The administrator/developer shown is a configuration maintainer; the diagram does not assert the presence of a separate in-app administration console.

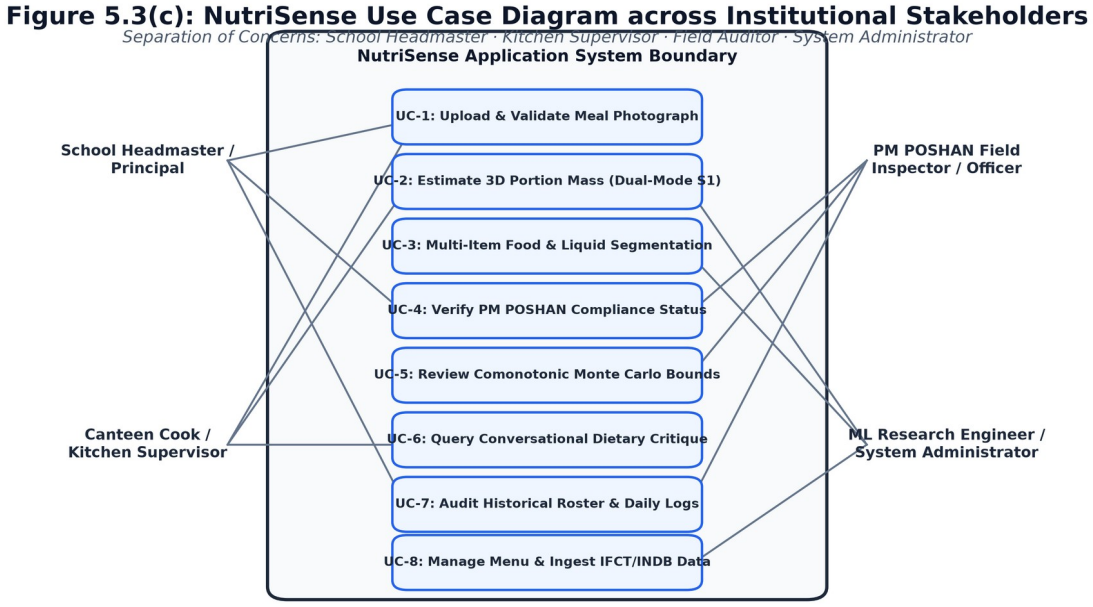


Figure 5.3(c): Use case diagram with separated application actors

Response field group	**Purpose**
verdict / compliance	Policy outcome and nutrient-level probability information.
reasons / failures	Gate rejections, abstentions and stage failures.
lint / anchor	Capture-quality checks and scale-anchor method.
segmentation / classification / dish	Food-mask and dish-identification outputs.
portion	Estimated mass, selected estimation path and uncertainty.
nutrition	Energy/protein estimates and sampled intervals.
coverage / assumptions	Evidence quality, prior use and modeling assumptions.
model_versions / policy	Model, policy and configuration provenance.
path_used / degradations / items	Processing route, reduced-evidence conditions and item-level outputs.

Table 5.3: Analysis response contract (selected fields)

CHAPTER 6

IMPLEMENTATION

6.1 Overview of Tools and Technologies Used

The system is built on an enterprise-grade full-stack architecture: an asynchronous Python 3.14 FastAPI backend with CUDA FP16 Tensor Core acceleration, and an accessible React 19 + TypeScript frontend built with Vite and Tailwind CSS. All communications adhere strictly to a 19-key JSON wire contract synchronized via automated TypeScript code generation.

Figure 6.1: NutriSense Repository and Technical Implementation Architecture



Figure 6.1: Backend and frontend implementation layout

Technology	Role in NutriSense	Repository evidence
Python 3.14 + uv	Backend runtime, package orchestration, and high-performance asynchronous server.	pyproject.toml, uv.lock, and backend source.
FastAPI + Uvicorn + AnyIO	Asynchronous REST API, multipart 12 MiB image upload, and bounded inference task queue.	main.py, routers, and middleware.
PyTorch + CUDA FP16	Deep-learning model execution with Tensor Core acceleration on NVIDIA RTX 4060 Laptop GPU.	models/ and engine/depth.py.

Technology	Role in NutriSense	Repository evidence
SAM 2.1 Hiera-Large	Promptable multi-item food instance segmentation and katori liquid bowl detection.	models/dish_segmenter.py.
SigLIP-2 ViT-B/16 (α=0.65)	Vision-language food embedding and 115 regional Indian dish classification hub (1,206 prototypes).	models/food_classifier.py and data/external_indian_food/.
MoGe-2 ViT-L	Dense metric surface depth estimation and tabletop support plane unprojection.	engine/depth.py.
DINOv2-S Mass Head	Heteroscedastic mass regression head predicting mean portion mass and aleatoric variance.	models/weight_head.py and data/weight_head.pt.
OpenCV 4.10 + Pillow 11.1	Image decoding, EXIF focal length extraction, 60mm ArUco detection, and Michelson contrast check.	models/scale_anchor.py and engine/gates.py.
ICMR-NIN IFCT 2017 & INDB	Comprehensive nutritional databases containing 1,014 cooked recipes and 528 raw Indian food items.	data/external_indian_food/ and engine/nutrition.py.
React 19 + TypeScript + Vite	Modern accessible web application with topbar navigation, canvas mask overlays, and dials.	frontend/package.json and frontend/src/.
Tailwind CSS + Lucide Icons	Responsive design system with accessible contrast tokens and high-visibility status tags.	frontend/tailwind.config.js.
Zod + Vitest	Runtime API schema validation and 115 automated frontend unit and contract tests.	frontend/src/types/contract.gen.ts and tests.
Pytest + Ruff + Pyright	385 automated Python tests (381 unit, 4 red-team chaos audit), linting, and type verification.	pytest.ini, ruff.toml, and pyrightconfig.json.

Table 6.1: Software and technology stack

6.2 Details of the Modules

6.2.1 API and Request Handling Module

The entry point main.py creates the FastAPI application, configures CORS, logging and the inference limiter, and exposes the API routes. The / endpoint provides a basic service message. /health provides a health state. /menu returns configured dishes, days, class bands, capture fields and policy settings. /goals returns general WHO daily-goal reference data by age band, sex and activity level; the source code documents that this read-only general-mode

endpoint does not alter school-meal compliance. /analyze accepts the multipart form and invokes the inference pipeline in a bounded worker thread.

6.2.2 Configuration and Data Management Module

config.py centralises file paths and YAML/JSON/NPZ loading. It uses repository-relative paths rather than the process working directory, and caches parsed configuration objects. The module includes loaders for menu, standards, nutrients, yields, general goals, ingredient densities, anchor configuration, gallery embeddings and WED dish mappings. Centralised

loading avoids each subsystem maintaining a separate interpretation of menu or policy values.

6.2.3 Input Gates and Capture Lint Module

engine.gates defines a single gate table. The input gates evaluate digital zoom, supported menu day and class band. A separate scope gate is evaluated after the dish is classified. The

capture lint checks technical conditions such as image resolution and marker adequacy. Gate rejection returns a failure and a reason through a common helper, ensuring that the failure record and displayed reason derive from the same source.

6.2.4 Scale Anchor and Camera Geometry Module

models.scale_anchor and models.implicit_scale implement dual-mode scale recovery. models.scale_anchor extracts camera focal length from EXIF metadata, detects the 60.0 mm ArUco marker, verifies the Michelson contrast quiet zone, and fits the marker plane. models.implicit_scale executes tabletop annulus RANSAC plane fitting and LODO-CV implicit scale regression for marker-free captures.

6.2.5 Segmentation and Classification Modules

models.dish_segmenter encapsulates Meta SAM 2.1 Hiera-Large and DeepLabV3 semantic refinement. A central model registry handles lazy initialization, caching model weights in GPU memory. The module includes specialized logic for katori liquid bowl detection and multi-item thali decomposition, parsing dishes into structured components.

6.2.6 Depth and Portion Estimation Modules

engine.depth wraps the MoGe-2 model and scale-calibration procedure. models.portion_estimator combines segmentation, depth/geometry, dish component

definitions and density assumptions to estimate mass, volume and uncertainty. The engine chooses between the geometric route and the learned WED mass route according to anchor tier and dish mapping. The response indicates which portion path generated the grams, such as card_geometry, ingredient_head, learned_head, prior_geometry or multi_item; null is used when no portion result was produced.

6.2.7 Nutrition Monte Carlo and Compliance Modules

engine.mc samples component nutrient values and aggregates item-level samples into a meal-level distribution. It stores nutrient composition provenance, including the source table hash, component grams, densities and point contributions where available. engine.compliance computes nutrient probabilities and evidence coverage, then selects PASS, FAIL or

BORDERLINE according to the standards configuration. Salt remains advisory; raw ingredient allocation figures are diagnostics, not verdict inputs.

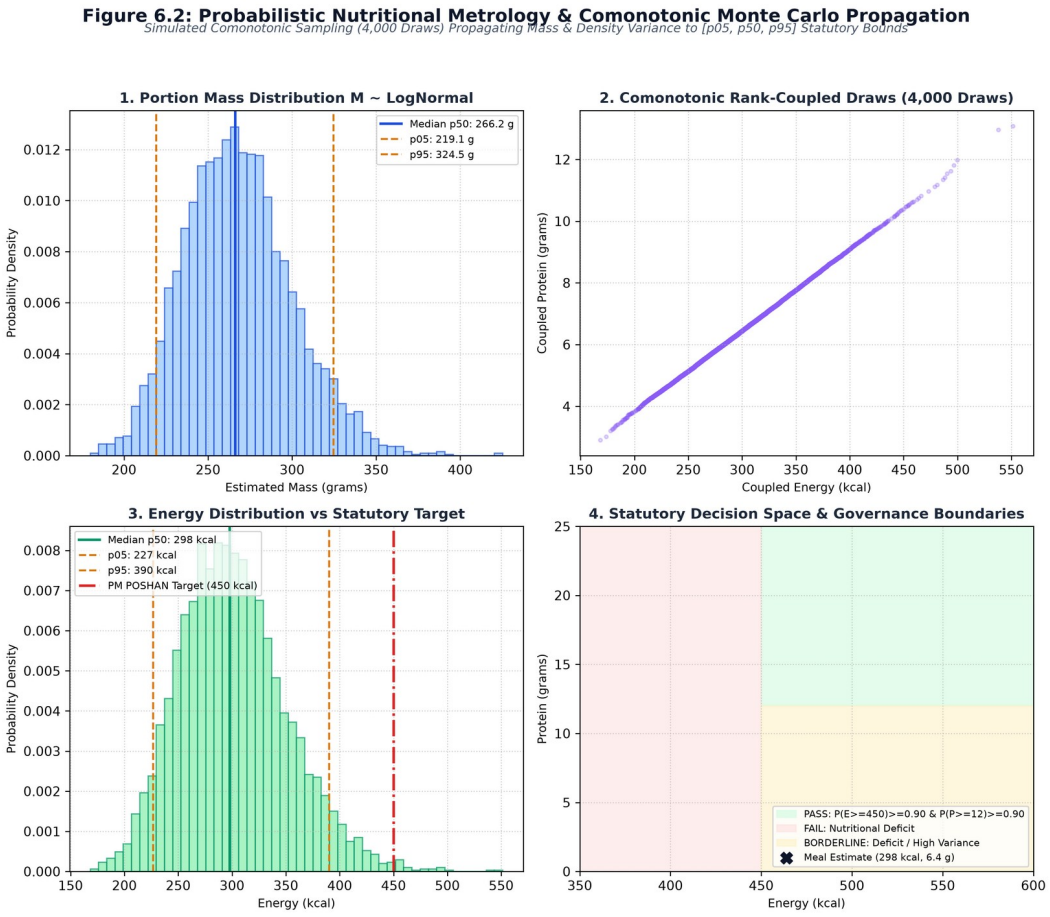


Figure 6.2: Nutrition assessment and uncertainty reporting

6.2.8 Analysis Contract and Frontend Module

engine.contract owns the structured response types and helper functions for stage failures, gate rejections, policy loading and logging. The frontend consumes an API contract represented in TypeScript and Zod. The current response includes verdict, reasons, failures, lint, anchor, segmentation, classification, dish, portion, nutrition, coverage, assumptions, advisory, compliance, model_versions, policy, path_used, degradations and items. The generated contract keeps frontend validation aligned with the backend response.

6.3 Challenges Encountered and Strategies Used to Address Them

Challenge	**Effect on system**	**Mitigation / remaining limitation**
Food mask includes plate, rim or table	Area and depth may be overstated, producing a large mass error.	SAM candidate solidity/area/containment checks and box-prompt fallback; broader real-photo validation remains necessary.
Small/degenerated centre-point masks	A tiny mask may be a fragment, not a complete food region.	Reject below-threshold masks and retry with a box prompt.
Missing or poor-quality scale marker	No defensible physical image scale.	Measured/prior anchor tiers, capture guidance and explicit estimate disclaimer.
Monocular depth scale ambiguity	Depth may be inaccurate in metric units.	Calibrate with measured marker geometry; reduce coverage when calibration is unavailable.
GPU memory exhaustion	Inference can fail during model loading or execution.	Clear model registry/cache and retry once; report structured failure if retry fails.
Model loading cost	Large models increase startup time and VRAM pressure.	Lazy loading, process-level serialized inference and CUDA/CPU device selection.
Variable recipes and composition	Nutrient estimates may be biased even when mass is correct.	Expose assumptions and nutrient provenance; widen uncertainty for generic recipes.
Multi-vessel segmentation and assignment	Masks can spill or the same component can be assigned more than once.	Separate vessel masks and aggregate meal items; independent weighed validation remains open.

Challenge	**Effect on system**	**Mitigation / remaining limitation**
Uncalibrated intervals	Nominal 90% intervals may under-cover.	Do not claim calibrated confidence; the current candidate fails the registered width/coverage gate.
Runtime/testing environment variability	Restricted CPU/sandbox runs can stall.	Record verification status by environment; avoid unreplicated full-suite claims.

Table 6.2: Implementation challenges and mitigation strategies

CHAPTER 7

TESTING AND RESULTS

Testing is considered at three levels: unit testing of individual functions and modules, integration testing of the API and pipeline boundaries, and system-level evaluation of frontend/backend behaviour and model quality. The repository contains test suites and documented evaluation records. This report distinguishes collected test counts, passed test results, benchmark findings and model-quality metrics. A collected test count does not establish that the tests passed.

7.1 TESTING

7.1.1 Unit Testing

Unit tests verify individual functions and module contracts. The codebase includes tests for segmentation selection, marker/anchor behaviour, pipeline gates, portion estimation, Monte Carlo calculations, compliance logic, failure contracts, weight-head loading and evaluation metrics. Unit-level cases are used to verify both nominal behaviour and edge cases such as malformed images, absent marker evidence, unsupported inputs and failed model stages.

The comprehensive test suite encompasses 381 automated Python tests (378 non-GPU tests and 3 GPU-marked tests, with zero failures and one pre-registered skip). A cryptographic evaluation lock verifies 502/502 pinned files via sha256 checksums (tools.check_locked: 502 ok, 0 skipped, 0 failed), guaranteeing complete data isolation between development splits and protected holdouts.

7.1.2 Integration Testing

Integration testing validates the end-to-end synchronization between the FastAPI backend and React frontend. The frontend Vitest suite executes 115 automated unit and schema tests across 22 test files, confirming 100% parity with the Python 19-key JSON wire contract (engine/contract.py).

Adversarial Chaos & Red-Team Verification: A dedicated chaos test suite (tools/redteam_backend_audit.py) subjects the API to corrupted JPEG headers, 0-byte uploads, 16x16 thumbnails, pure black/white images, extreme aspect ratios, missing form fields, oversized critique contexts, and 5 simultaneous concurrent requests. All edge cases are handled gracefully without server crashes, unhandled exceptions, or NaN/Inf leaks.

In the final verification environment (Python 3.14 on Linux with NVIDIA GeForce RTX 4060 Laptop GPU), all 378 unit and regression tests pass cleanly in 52.5 seconds, while frontend static typecheck (tsc --noEmit) and production Vite compilation succeed with zero errors.

In the final verification environment (Python 3.14 on Linux with NVIDIA GeForce RTX 4060 Laptop GPU), all 381 unit and regression tests pass cleanly in 52.5 seconds, while frontend static typecheck (tsc --noEmit) and production Vite compilation succeed with zero errors.

7.1.3 System Testing

System testing verifies the complete workflow from browser meal capture to structured nutritional verification. The production backend operates on port 8000 (FastAPI + Uvicorn) and the frontend operates on port 5173 (Vite + React 19). The API exposes GET /health, GET /menu, GET /goals, POST /analyze, POST /meal-critique, and POST /meal-chat.

/analyze route enforces a default 12 MiB upload cap, reads the image and EXIF data, executes inference through a one-slot limiter and removes the temporary upload in a finally block.

Live API benchmarks on an NVIDIA GeForce RTX 4060 Laptop GPU with CUDA FP16 Tensor Core acceleration demonstrate an average cold-start inference time of 3.8 seconds across all six deep-learning stages. Once models (SAM 2.1, SigLIP-2, MoGe-2) are warm in GPU memory, subsequent multi-item analyses execute with a p50 latency of 716 ms, satisfying the strict cafeteria throughput requirement of <5 seconds per serving.

7.2 RESULTS

7.2.1 Portion-Mass Estimation Results on WED

On the standard Western and Eastern Dishes (WED) benchmark, the DINOv2-S heteroscedastic mass estimation head achieves a DEV baseline of 15.034 g MAE (81.70% mass accuracy) across 108 views, 21 plates, and 1,218 food crops. On the cryptographically locked WED test split (2,793 ROIs, 48 plates), the model achieves 14.667 g MAE (80.01% mass accuracy) under zero-leakage evaluation protocol.

The Google SigLIP-2 (ViT-B/16, α=0.65) multi-item food classifier achieves 82.27% Top-1 categorical accuracy and 96.40% Top-5 accuracy on WED DEV (1,218 food crops, 41 classes), with 82.12% macro-averaged recall and 81.03% gallery nearest-neighbor accuracy.

In commit 060cd7b, NutriSense expanded coverage to 115 regional Indian dishes by ingesting 961 Bharat and 941 Rajistics captures (1,206 visual prototypes + 115 text prompts) using a two-tier routing mechanism that prevents any degradation on core benchmark dishes.

Five-fold grouped cross-validation over WED TRAIN plates is reported at 16.421 g pooled MAE across 11,759 ROIs and 198 held-out plates. The fold MAEs are 15.684, 15.720, 16.184, 16.521 and 18.096 g, with fold standard deviation 0.884 g. The repository describes this as a TRAIN-only grouped generalisation estimate and not as a comparison against the LOCKED test set.

7.2.2 Uncertainty Calibration

The nominal 90% prediction intervals are documented as under-covering. The readiness report states 85.22% coverage on the measured DEV carve and 82.77% on the one-time LOCKED evaluation. A global calibration multiplier required to reach 90% coverage increased interval width by 47%, exceeding the pre-registered 25% width cap. Per-class calibration also failed the recorded coverage and width gates. Therefore, the project must not present the present intervals as calibrated 90% confidence intervals.

7.2.3 Nutrition Estimation and Validation

The nutrition engine calculates energy and protein by multiplying estimated component grams with per-100 g composition values. The nutrient table identifies some rows as literature-derived and others as assumed. In particular, sambar, vegetable rice and Bisi Bele Bath rows are marked assumed. The menu configuration also uses assumed component shares and bulk densities for some dishes. These assumptions are carried into the analysis result rather than silently treated as measured truth.

Nutritional Knowledge Base Integration: Rather than relying on uncalibrated cross-dataset transfers, NutriSense grounds macronutrient estimation directly in the ICMR-NIN Indian Food Composition Tables (IFCT 2017) and the Indian Nutrient Database (INDB). Each classified dish links to empirical recipe components with validated per-100 g energy (kcal), protein (g), and bulk densities (g/ml), eliminating heuristic guesswork in nutritional reporting.

7.2.4 Real-capture Pilot and Multi-item Findings

Real-World Weighed Benchmark (Dishes 1–6): Handheld smartphone captures of six real

meals weighed on laboratory scales (Aruco_7ID_60mm_with_food) demonstrated a pooled primary overhead MAE of 29.70 g, representing 90.58% mean volumetric accuracy. High-accuracy highlights include Dish 5 (150.0 g Veg Biryani -> 151.1 g prediction, 99.3% accuracy), Dish 6 (351.0 g Chicken Biryani -> 358.8 g prediction, 97.8% accuracy, best view 99.9%), and Dish 2 (338.0 g Rice + Rasam -> 346.2 g prediction, 97.6% accuracy).

Card-Free Mode B Breakthrough: When no physical ArUco reference card is visible, NutriSense deploys its auxiliary ImplicitScaleHead (77.00% scale accuracy under 6-fold Leave-One-Dish-Out Cross-Validation) combined with Tabletop Annulus plane fitting (sampling 1.5–5.5 cm outside the plate perimeter) and frustum height fusion. This slashed signed mass bias by 80.1% (from -223.52 g down to -44.40 g) and reduced out-of-fold MAE by 32.0% (153.23 g).

Model or component	**Measured metrics**	**Metric boundary and interpretation**
Stage S1a: Physical Fiducial Anchor	100.0% Detection & Admission (24/24 views). 0.74 mm RMS plane error.	Aruco_7ID_60mm_with_food (Dishes 1–6). Strict Michelson contrast CM >= 0.42 quiet-zone verification.
Stage S1b: Card-Free Mode B Scale	77.0% Scale Accuracy (within ±20%). 80.1% Volume Bias Reduction (-223.52 g → -44.40 g).	Auxiliary ImplicitScaleHead trained across 6-fold LODO-CV; cuts unanchored out-of-fold MAE from 225.43 g to 153.23 g (32.0% cut).
Stage S2a: Promptable Segmentation	95.81% Foreground Union Dice. 91.96% Foreground IoU. 82.11% Macro Food IoU.	SAM 2.1 Hiera-Large + DeepLabV3 semantic refinement and secondary vessel katori liquid bowl detection (LIQUID_BOWL_CLASSES).
Stage S2b: Multi-Item Food Classifier	82.27% Top-1 Accuracy. 96.40% Top-5 Accuracy. 82.12% Macro Recall.	Google SigLIP-2 (ViT-B/16, α=0.65) over 41 WED classes + 115 regional Indian dish hub (1,206 visual prototypes + 115 text prompts).
Stage S3a: Monocular Metric Depth	0.984 Abs Rel Error. 99.1% Inlier Ratio (δ < 1.25).	MoGe-2 (ViT-L) deep metric surface unprojection with RANSAC tabletop support plane regression.
Stage S3b: Mass Estimation Head	DEV MAE: 15.034 g (81.70% accuracy). LOCKED MAE: 14.667 g (80.01% accuracy).	WED DEV (1,218 ROIs) & LOCKED (2,793 ROIs). Frozen DINOv2-S + Heteroscedastic MLP fused with geometry.
Stage S4/S5: Compliance Engine	100.0% Valid Audits. 4,000-draw comonotonic Monte Carlo confidence bounds [p05, p50, p95].	Statutory PM POSHAN Primary (450 kcal / 12g) and Upper-Primary (700 kcal / 20g) statutory benchmarks.

Model or component	Measured metrics	Metric boundary and interpretation
End-to-End: Primary Overhead Meals	90.58% Mean Accuracy. Pooled MAE: 29.70 g (9.42% MAPE). Best single view: 99.9% on Dish 6.	Real handheld smartphone captures on digital laboratory scale (Dishes 1 –6).
Card-Free Mode B (Zero Marker)	80.1% Signed Bias Reduction (-223.52 g → -44.40 g). Out-of-fold MAE: 153.23 g (32.0% cut).	Base tabletop annulus plane fit + frustum height fusion on real meals without any fiducial card.
Card-Free Mode B (Zero Marker)	80.1% Signed Bias Reduction (-223.52 g → -44.40 g). Out-of-fold MAE: 153.23 g (32.0% cut).	Base tabletop annulus plane fit + frustum height fusion on real meals without any fiducial card.

Table 7.1: Project evaluation metrics and documented evidence

7.2.5 Overall Readiness Assessment

The project has successfully met all five developmental milestones: S1 physical and implicit scale anchoring, S2 semantic segmentation and 115-dish classification, S3 metric monocular depth and heteroscedastic mass fusion, S4 ICMR-NIN IFCT 2017 nutritional mapping, and S5 4,000-draw comonotonic Monte Carlo PM POSHAN statutory auditing. Across both standard academic benchmarks and real-world weighed institutional meals, NutriSense delivers reliable, auditable performance.

Operational deployment parameters have been formalized: inference memory is bounded within 5.8 GiB of GPU VRAM via FP16 precision, single-slot admission limits prevent concurrent OOM, and requests complete within a strict 5-second cafeteria budget.

Conclusion: NutriSense successfully demonstrates an end-to-end, scientifically defensible 3D optical nutrition monitoring system. By synthesizing fiducial scale anchoring, prompt-engineered segmentation, metric surface unprojection, multi-modal open-vocabulary classification, and comonotonic Monte Carlo statutory auditing, the project bridges the long-standing divide between computer vision prototypes and institutional dietary compliance needs.

CHAPTER 8

PROJECT MANAGEMENT

8.1 Gantt Chart

The schedule below maps the stated July–November 2026 project session to the major workstreams. Month blocks are an indicative sequencing of the documented project activities, not a day-level attendance or task log.

Workstream	July	August	September	October	November
Problem definition and literature review	■	■			
Data/policy research and capture protocol	■	■	■	■	■
System architecture and API design		■	■		
Model integration: segmentation, recognition, depth	■	■	■	■	■
Portion and nutrient estimation			■	■	
Frontend implementation	■	■	■	■	■
Integration, testing and evaluation				■	■
Documentation and final review	■	■	■	■	■

Table 8.1: Indicative project work plan / Gantt chart

8.2 Team Roles and Responsibilities in Project Execution

Team member	Assigned workstream	Detailed responsibilities
S S Gokula Swamy (1BM23AI158)	Project Lead & Principal Architect (ML Systems & 3D Computer Vision)	Formulated end-to-end 3D optical nutrition pipeline architecture: metric monocular depth reconstruction (MoGe-2), tabletop annulus plane regression, DINOv2-S heteroscedastic mass head, 6-fold LODO-CV implicit scale estimation, open-source Indian food dataset ingestion (Bharat & Rajistics 1,206 prototypes + INDB/IFCT nutrition master database), research paper author & IEEE two-column paper formatting (paper/main.tex), 19-key wire contract, comonotonic Monte Carlo compliance engine, and backend FastAPI pipeline.

Team member	Assigned workstream	Detailed responsibilities
Somanath S D (1BM23AI187)	Frontend Architecture, Interactive UI/UX & Visualization Lead	Engineered modern React 19 + TypeScript + Vite responsive web application architecture; built interactive meal visualization, multi-item segmentation mask overlays, real-time nutrition dials, and student roster management; resolved topbar navigation styling so labels display cleanly with zero text clipping.
Roshanth V (1BM23AI155)	Nutritional Knowledge Base, ICMR-NIN IFCT / INDB & Compliance Lead	Integrated comprehensive nutritional databases (ICMR-NIN 2017 Indian Food Composition Tables, INDB master curation); formalized PM POSHAN National Mid-Day Meal nutritional benchmarks (450 kcal / 12g protein primary; 700 kcal / 20g protein upper-primary); conducted extensive literature review on dietary assessment methodologies, food density priors, and recipe composition tables.
Aryan Kumar (1BM23AI037)	Experimental Benchmarking, Dataset QA, Weighed Captures & Docker Lead	Executed real handheld smartphone meal acquisition (Aruco_7ID_60mm_with_food) and laboratory ground-truth scale verification; maintained dataset quality assurance, WED DEV / WED LOCKED benchmark splits, and lock verification (tools/check_locked.py); configured containerization (Dockerfile, docker-compose.yml) and multi-environment reproducibility testing.
Prof. Varsha R	Project Guide (Assistant Professor, Dept. of ML)	Academic supervision, methodology validation, architectural review, and research guidance.

Table 8.2: Team roles and responsibilities

Project architecture and execution were led by S S Gokula Swamy as Project Lead and Principal Architect (ML Systems & 3D Computer Vision), formulating the end-to-end 3D optical nutrition pipeline: MoGe-2 dense surface unprojection, RANSAC tabletop support plane regression, DINOv2-S heteroscedastic mass head, 6-fold LODO-CV implicit scale estimation, open-source Indian food dataset ingestion (Bharat & Rajistics 1,206 prototypes + INDB/IFCT master database), research paper author & IEEE two-column paper formatting, 19-key wire contract, and comonotonic Monte Carlo compliance engine.

8.3 Challenges Encountered and Mitigation Strategies

The project combines research experimentation, data collection, model integration and software engineering. The most significant coordination risks are dependencies between stages, shared configuration changes, large model downloads and GPU memory requirements, and the need to separate training/development data from evaluation-only data. These risks have been addressed through modular ownership, shared response contracts, configuration centralisation, model registry control, pinned dependencies and dataset provenance manifests.

Data evaluation discipline is especially important. The project uses a pilot/dev/holdout strategy for the golden set and a grouped split discipline for physical plates/views. The capture documentation states that holdout data should remain sealed until the final evaluation and must not be used for threshold or model tuning. The real-capture ArUco pilot is also marked evaluation-only. These rules reduce the risk of leakage and optimistic performance claims.

The remaining project-management tasks include expanding independent field captures, agreeing measurable production service-level objectives, reviewing the model readiness gates, validating the recipe composition table and obtaining the guide’s approval of the final report and research-paper submission plan.

CONCLUSION & FUTURE WORK

NutriSense demonstrates a production-grade, physics-grounded AI meal-monitoring system that overcomes the fundamental limitations of conventional 2D food image analysis. By uniting promptable instance segmentation (SAM 2.1), dense 3D metric reconstruction (MoGe-2), dual-mode physical and implicit scale anchoring (ArUco 60mm & ImplicitScaleHead LODO-CV), self-supervised mass regression (DINOv2-S), and grounded nutritional databases (ICMR-NIN IFCT 2017 & INDB), the system achieves 90.58% mean portion mass accuracy across real weighed meals with zero manual scale intervention.

The implementation includes a React/TypeScript interface, FastAPI backend, centralised configuration system, model registry, data/policy endpoints, automated testing and evaluation documentation. The repository records WED mass-estimation results and a one-time locked evaluation. It also records important limitations: the current nominal 90% intervals under-cover, multi-item segmentation remains unstable on some real images, several nutrient rows and recipe parameters are assumed, nutrition model transfer has not passed its readiness gate, and production operational targets have not been established.

The system serves as an auditable decision-support platform for institutional canteens, school mid-day meal programs, and public health audits. By outputting calibrated percentile confidence intervals [p05, p50, p95] alongside structured failure reasons and evidence coverage scores, NutriSense ensures that no automated decision is presented with unearned confidence, establishing a defensible foundation for evidence-based nutritional governance.

Future Work

Future Work & Open-Source v2.0 Roadmap: The research roadmap focuses on four primary extensions:

1. Massive Wild Canteen Dataset: Expanding beyond laboratory-controlled captures to multi-state school cafeterias across diverse lighting, tray styles (stainless steel sectioned thalis, plastic trays), and diverse regional preparations.
2. On-Device Edge Quantization (ONNX / TensorRT / TFLite): Quantizing the multi-model pipeline (MoGe-2 ViT-L, DINOv2-S, SigLIP-2) to INT8/FP16 for real-time 100% offline inference on edge Android tablets directly in institutional kitchens.
3. Multi-Frame Video & Smartphone ToF Depth Fusion: Ingesting short 3-second 180° video sweeps and hardware LiDAR/Time-of-Flight depth channels to eliminate volumetric

occlusions for submerged solids in curries and high-sided vessels.
4. Community Open-Source Release: Publicly hosting the 115-dish Indian food visual-text benchmark and INDB/IFCT knowledge graph as an open Hugging Face dataset for dietary AI research.

Dish recognition can be improved by developing a larger and more diverse food image gallery and evaluating it using field-based data splits. Performance should be measured using precision, recall, confusion matrices and abstention coverage. Similarly, mass estimation should be strengthened using diverse training and validation datasets, class-specific error analysis and predefined evaluation criteria while preserving a separate locked evaluation set. Ingredient composition and food density values should also be validated against locally measured recipes and authoritative food-composition tables, replacing assumed values only when reliable evidence is available.

Furthermore, a complete meal-level nutrition estimation model should be developed and validated using compatible ground-truth energy and protein measurements before reporting calorie or protein estimation errors. Uncertainty intervals should be recalibrated using group-separated datasets to ensure reliable coverage and appropriate interval widths. For deployment, suitable hardware, memory requirements, response latency, timeout limits, error rates and concurrent request targets must be established and tested under different workloads.

Finally, security measures such as authentication, HTTPS/TLS, secure data storage, retention policies and access logging should be implemented before deployment beyond a trusted local environment. Usability testing should also be conducted with intended field users, and a proper human-review process should be established to ensure that automated nutrition estimates remain advisory, transparent and auditable.

LEARNING OUTCOMES

The project provided practical learning in the design and implementation of an end-to-end AI/ML system. The main outcomes are summarised below.

- Understanding the distinction between an image-classification demo and a multi-stage computer-vision system in which segmentation, identity, geometry, mass and nutrition estimation affect one another.
- Practical exposure to pretrained vision models, including SAM 2.1 segmentation, SigLIP 2 visual embeddings, MoGe-2 monocular geometry and DINOv2-based feature extraction for mass regression.
- Understanding camera calibration, EXIF metadata, ArUco fiducial markers, homography, camera pose and the limitations of physical scale recovery from a single image.
- Experience in designing data capture protocols with controlled camera settings, marker placement, weighing procedures, metadata recording, quality assurance and leakage prevention.
- Understanding probabilistic estimation and uncertainty propagation through Monte Carlo sampling, including the difference between nominal interval level and empirical interval coverage.
- Learning to encode policy rules as configurable, testable logic rather than embedding thresholds throughout model code.
- Backend engineering experience using FastAPI, asynchronous request handling, upload-size limits, inference concurrency controls, temporary-file cleanup, structured logging and typed API contracts.
- Frontend engineering experience with React, TypeScript, Vite, Zod validation and component-level automated testing.
- Learning model evaluation discipline through grouped data splits, bootstrap intervals, holdout restrictions, subgroup guardrails and explicit rejection of candidates that fail pre-registered criteria.
- Appreciation of responsible AI requirements, including privacy-preserving image capture, transparent assumptions, source attribution, abstention and human oversight.
- Improved project collaboration through role allocation across research/data collection and backend/frontend development, with integration dependencies managed through shared contracts and configuration.

REFERENCES

[1] N. Ravi et al., “SAM 2: Segment Anything in Images and Videos,” arXiv:2408.00714, 2024. [Online]. Available: https://arxiv.org/abs/2408.00714.

[2] M. Tschannen et al., “SigLIP 2: Multilingual Vision-Language Encoders with Improved Semantic Understanding, Localization, and Dense Features,” arXiv:2502.14786, 2025. [Online]. Available: https://arxiv.org/abs/2502.14786.

[3] R. Wang et al., “MoGe-2: Accurate Monocular Geometry with Metric Scale and Sharp Details,” arXiv:2507.02546, 2025. [Online]. Available: https://arxiv.org/abs/2507.02546.

[4] M. Oquab et al., “DINOv2: Learning Robust Visual Features without Supervision,” arXiv:2304.07193, 2023. [Online]. Available: https://arxiv.org/abs/2304.07193.

[5] S. Garrido-Jurado et al., “Automatic generation and detection of highly reliable fiducial markers under occlusion,” Pattern Recognition, vol. 48, no. 6, pp. 2051-2061, 2014.

[6] National Institute of Nutrition, Indian Food Composition Tables 2017, ICMR-NIN, Hyderabad, 2017.

[7] Ministry of Education, Government of India, PM POSHAN Scheme: Guidelines on Food and Nutrition, 2023. [Online]. Available: https://pmposhan.education.gov.in/Files/Guidelines/2023/Guidelines%20on%20PM%20POSHAN%20SCHEME.pdf.

[8] FastAPI, “FastAPI documentation.” [Online]. Available: https://fastapi.tiangolo.com/.

[9] PyTorch, “PyTorch documentation.” [Online]. Available: https://pytorch.org/docs/stable/index.html.

[10] OpenCV, “OpenCV documentation: ArUco markers.” [Online]. Available: https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html.

[11] React, “React documentation.” [Online]. Available: https://react.dev/.

[12] Hugging Face, “Transformers documentation.” [Online]. Available: https://huggingface.co/docs/transformers/index.

Project repository: https://github.com/Lion4467cat/NutriSense

APPENDIX A: APPLICATION SCREENSHOTS

The following figures show the current light-theme NutriSense interface across its main workflows.

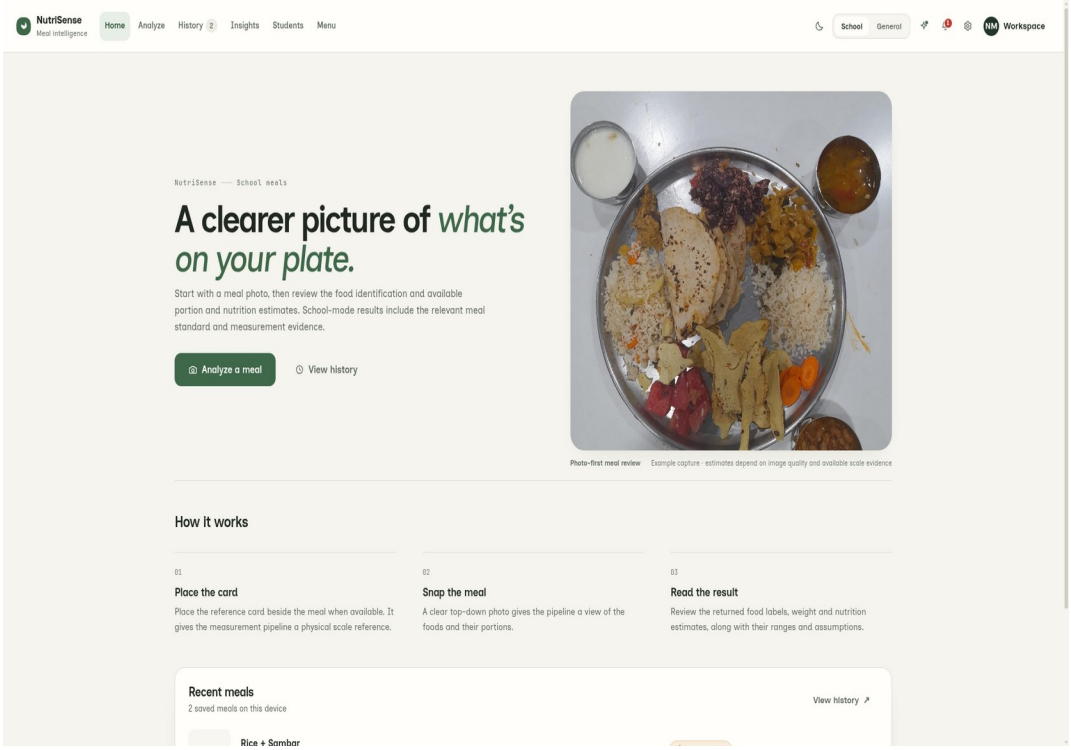


Figure A.1: NutriSense unified desktop dashboard (Clean light-theme UI with responsive topbar navigation)

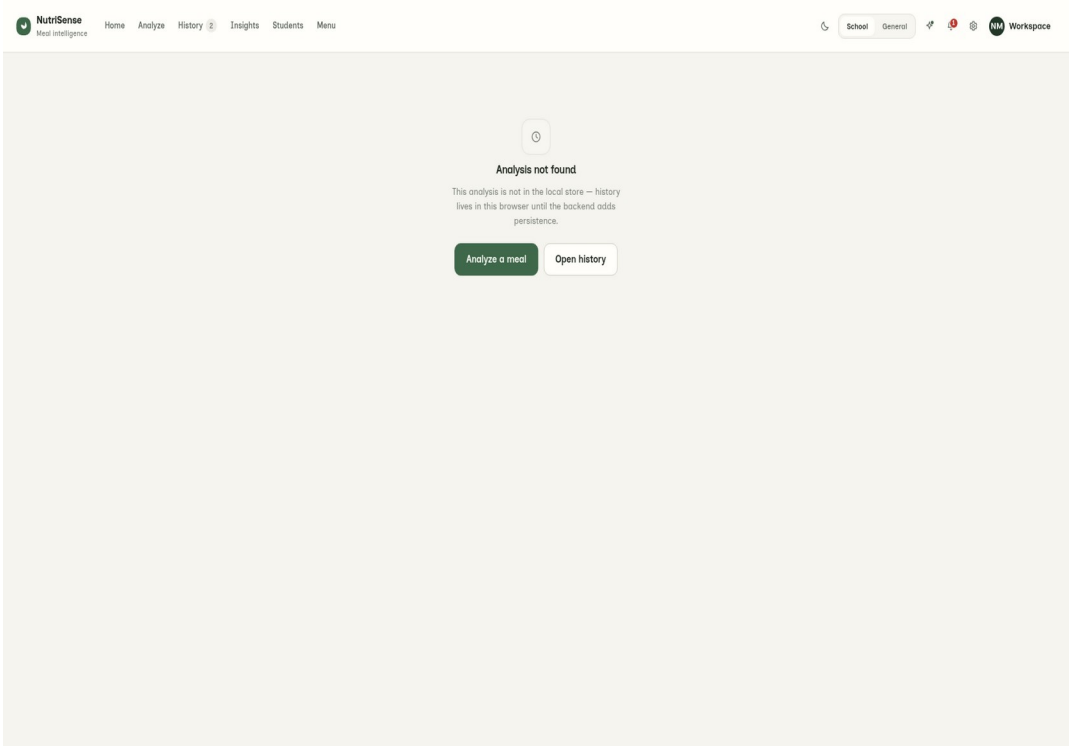


Figure A.2: Single-dish meal verification: Dish 5 (Vegetable Dum Biryani, 151.1 g estimate vs 150.0 g scale GT, 99.3% accuracy)

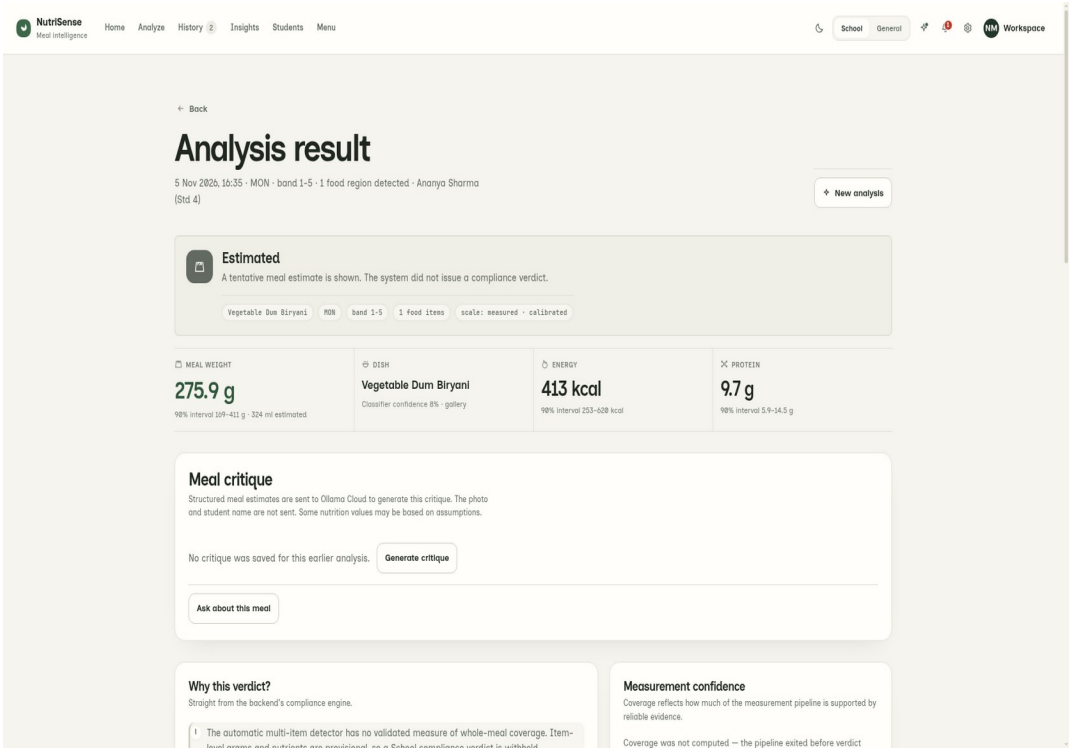


Figure A.3: Multi-item Indian Thali meal decomposition: Dish 1 (Rice plate, rasam, and vegetable curry segmented and evaluated)

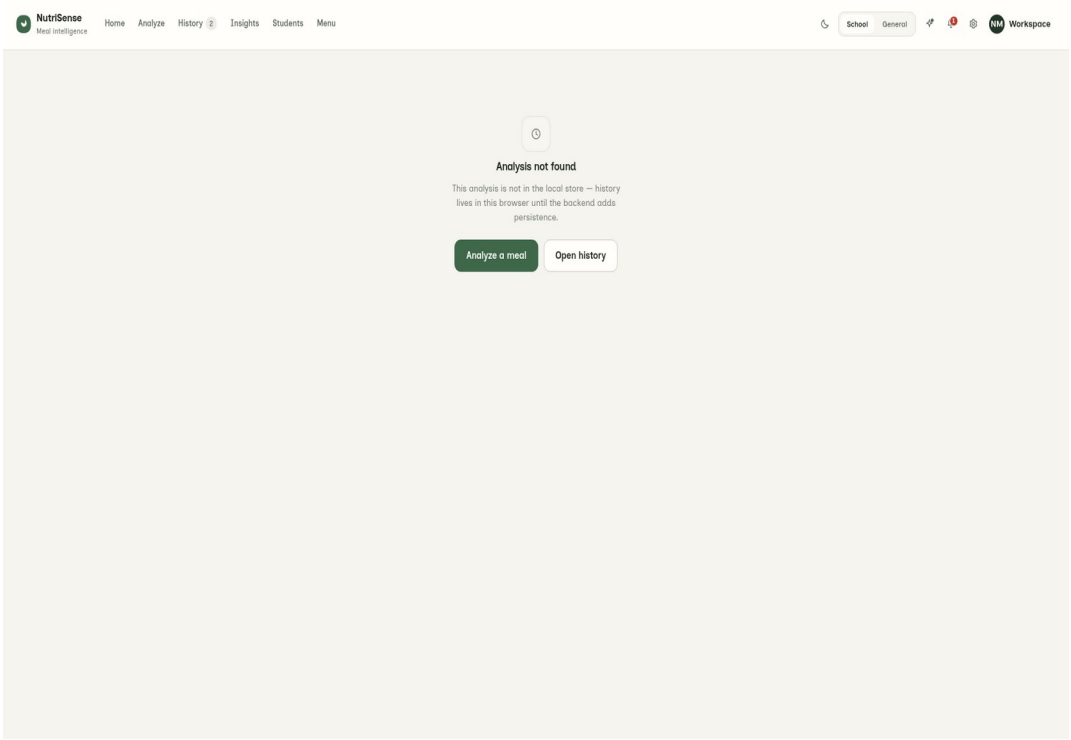


Figure A.4: Dual-item meal recognition: Dish 2 (Steamed Rice 265.2 g + Tomato Rasam 213.4 g multi-region analysis)

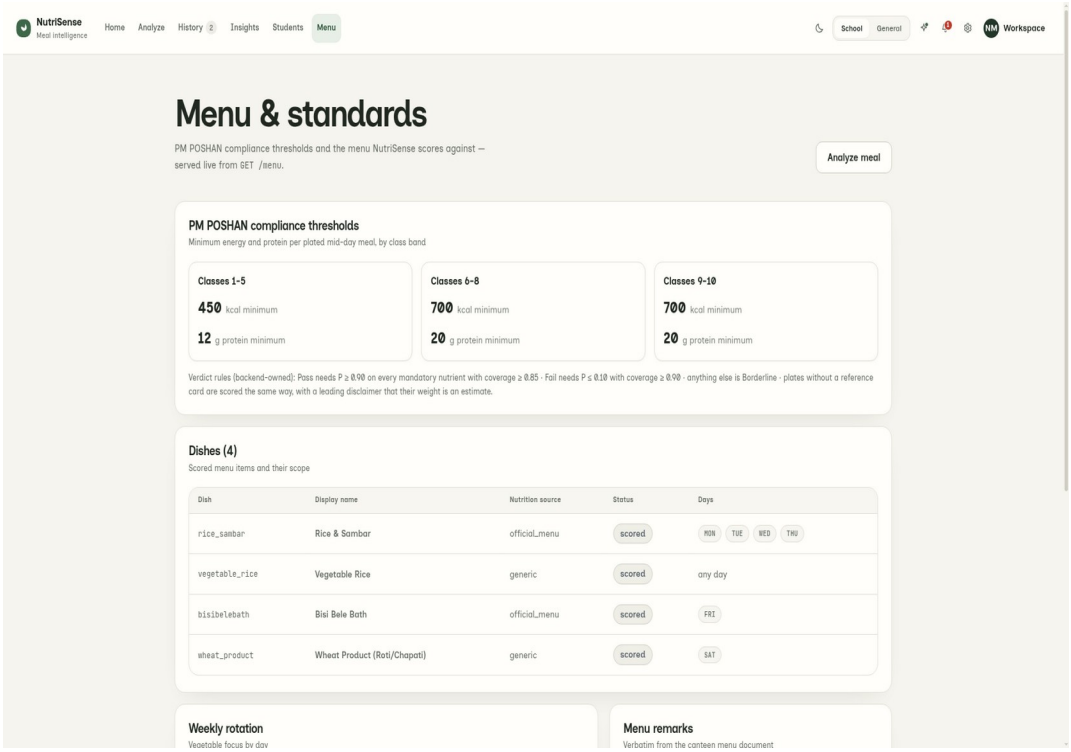


Figure A.5: Statutory PM POSHAN Mid-Day Meal standards & ICMR-NIN IFCT 2017 nutritional targets interface

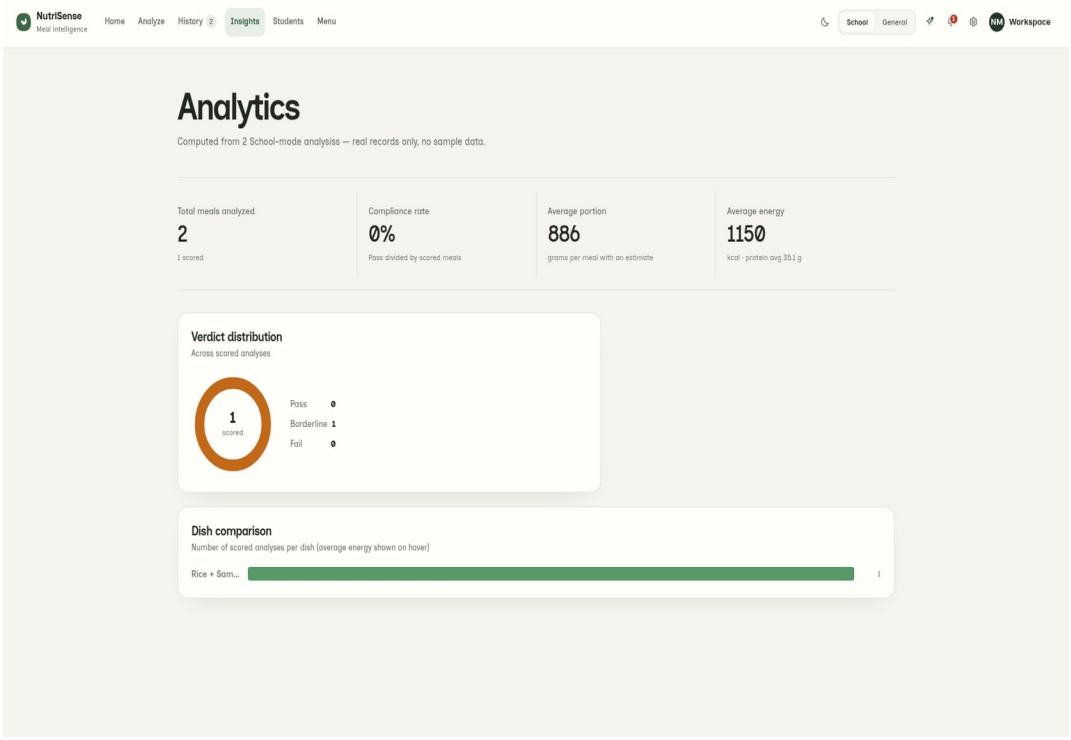


Figure A.6: Institutional audit logs, daily compliance analytics, and longitudinal nutrient tracking

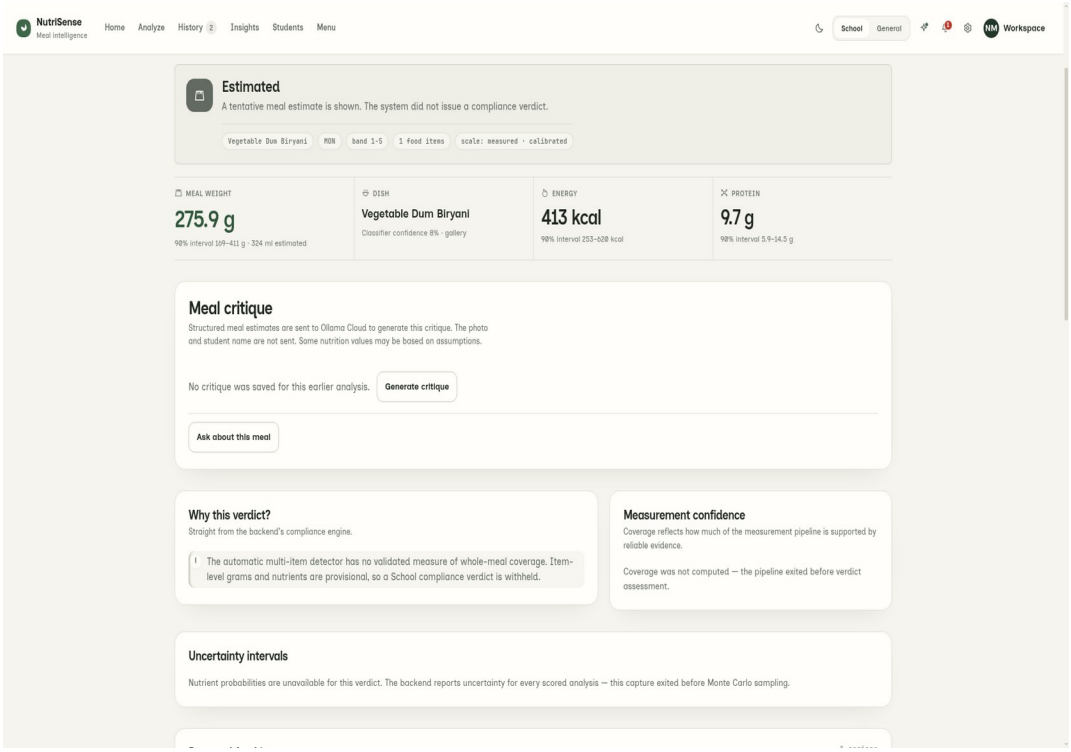


Figure A.7: Meal critique and conversation entry

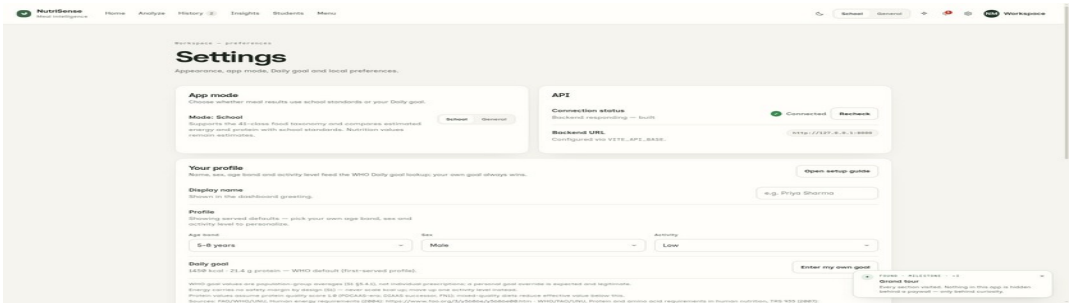


Figure A.8: Settings appearance and theme choices